Principia Softwarica: The Plan 9 Profilers

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
??

January 14, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **4**
- 1.1 Motivations . . . 4
- 1.2 The Plan 9 profiler: `prof` . . . 4
- 1.3 Other profilers . . . 4
- 1.4 Getting started . . . 4
- 1.5 Requirements . . . 4
- 1.6 About this document . . . 4
- 1.7 Copyright . . . 4
- 1.8 Acknowledgments . . . 5

2 Overview **6**
- 2.1 Profiler principles . . . 6
- 2.2 `prof` command-line interface . . . 6
- 2.3 Code organization . . . 6
- 2.4 Software Architecture . . . 6
- 2.5 Book structure . . . 6

3 Core Data Structures **7**

4 Conclusion **8**

A Debugging **9**

B Error Management **10**

C Utilities **11**

D Extra Code **12**
- D.1 `profilers/` . . . 12
  - D.1.1 `misc/time.c` . . . 12
  - D.1.2 `misc/kprof.c` . . . 14
  - D.1.3 `misc/prof.c` . . . 16
  - D.1.4 `misc/trace.c` . . . 24
  - D.1.5 `misc/tprof.c` . . . 39
  - D.1.6 `misc/stats.c` . . . 42
- D.2 `iostats/` . . . 70
  - D.2.1 `iostats/statfs.h` . . . 70
  - D.2.2 `iostats/globals.c` . . . 73
  - D.2.3 `iostats/iostats.c` . . . 74
  - D.2.4 `iostats/statsrv.c` . . . 85

Glossary 99

Index 100

References 100

Chapter 1

Introduction

The goal of this book is to present in full details the source code of a few profilers.

1.1 Motivations

Why a profiler? Because I think you are a better programmer if you fully understand how things work under the hood.

1.2 The Plan 9 profiler: `prof`

1.3 Other profilers

Here are a few profilers that I considered for this book, but which I ultimately discarded:

- Jon Bentley's toy profiler
- gprof
- oprofile
- perf

1.4 Getting started

1.5 Requirements

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

Chapter 2

Overview

2.1 Profiler principles

2.2 `prof` command-line interface

2.3 Code organization

2.4 Software Architecture

2.5 Book structure

Chapter 3

Core Data Structures

Chapter 4

Conclusion

Appendix A

Debugging

Appendix B

Error Management

Appendix C

Utilities

Appendix D

Extra Code

D.1 profilers/

D.1.1 misc/time.c

⟨*global* output 12a⟩≡ (14a)

```
char output[4096];
```

⟨*function* error 12b⟩≡ (14a)

```
static void
error(char *s)
{

    fprint(2, "time: %s: %r\n", s);
    exits(s);
}
```

⟨*function* main 12c⟩≡ (14a)

```
void
main(int argc, char *argv[])
{
    int i;
    Waitmsg *w;
    long l;
    char *p;
    char err[ERRMAX];

    if(argc <= 1){
        fprint(2, "usage: time command\n");
        exits("usage");
    }

    switch(fork()){
    case -1:
        error("fork");
    case 0:
        exec(argv[1], &argv[1]);
        if(argv[1][0] != '/' && strncmp(argv[1], "./", 2) &&
           strncmp(argv[1], "../", 3)){
            sprint(output, "/bin/%s", argv[1]);
            exec(output, &argv[1]);
        }
        error(argv[1]);
    }
```

```
    notify(notifyf);

    loop:
    w = wait();
    if(w == nil){
        errstr(err, sizeof err);
        if(strcmp(err, "interrupted") == 0)
            goto loop;
        error("wait");
    }
    l = w->time[0];
    add("%ld.%.2ldu", l/1000, (l%1000)/10);
    l = w->time[1];
    add("%ld.%.2lds", l/1000, (l%1000)/10);
    l = w->time[2];
    add("%ld.%.2ldr", l/1000, (l%1000)/10);
    add("\t");
    for(i=1; i<argc; i++){
        add("%s", argv[i], 0);
        if(i>4){
            add("...");
            break;
        }
    }
    if(w->msg[0]){
        p = utfrune(w->msg, ':');
        if(p && p[1])
            p++;
        else
            p = w->msg;
        add(" # status=%s", p);
    }
    fprint(2, "%s\n", output);
    exits(w->msg);
}
```

⟨*function* add 13a⟩≡ (14a)

```
void
add(char *a, ...)
{
    static int beenhere=0;
    va_list arg;

    if(beenhere)
        strcat(output, " ");
    va_start(arg, a);
    vseprint(output+strlen(output), output+sizeof(output), a, arg);
    va_end(arg);
    beenhere++;
}
```

⟨*function* notifyf 13b⟩≡ (14a)

```
void
notifyf(void *a, char *s)
{
    USED(a);
    if(strcmp(s, "interrupt") == 0)
        noted(NCONT);
    noted(NDFLT);
}
```

⟨misc/time.c 14a⟩≡
```
#include <u.h>
#include <libc.h>

⟨global output 12a⟩
void add(char*, ...);
void notifyf(void*, char*);

⟨function error 12b⟩
⟨function main 12c⟩
⟨function add 13a⟩
⟨function notifyf 13b⟩
```

D.1.2 misc/kprof.c

⟨constant PCRES 14b⟩≡ (16a)
```
#define PCRES 8
```

⟨struct COUNTER 14c⟩≡ (16a)
```
struct COUNTER
{
    char  *name;  /* function name */
    long time;  /* ticks spent there */
};
```

⟨function error (misc/kprof.c) 14d⟩≡ (16a)
```
static void
error(int perr, char *s)
{
    fprint(2, "kprof: %s", s);
    if(perr){
        fprint(2, ": ");
        perror(0);
    }else
        fprint(2, "\n");
    exits(s);
}
```

⟨function compar 14e⟩≡ (16a)
```
static int
compar(void *va, void *vb)
{
    struct COUNTER *a, *b;

    a = va;
    b = vb;
    if(a->time < b->time)
        return -1;
    if(a->time == b->time)
        return 0;
    return 1;
}
```

⟨function main (misc/kprof.c) 14f⟩≡ (16a)
```
void
main(int argc, char *argv[])
{
    int fd;
    long i, j, k, n;
```

```
char *name;
ulong *data;
vlong tbase;
ulong sum;
long delta;
Symbol s;
Biobuf outbuf;
Fhdr f;
Dir *d;
struct COUNTER *cp;

if(argc != 3)
    error(0, "usage: kprof text data");
/*
 * Read symbol table
 */
fd = open(argv[1], OREAD);
if(fd < 0)
    error(1, argv[1]);
if (!crackhdr(fd, &f))
    error(1, "read text header");
if (f.type == FNONE)
    error(0, "text file not an a.out");
if (syminit(fd, &f) < 0)
    error(1, "syminit");
close(fd);
/*
 * Read timing data
 */
fd = open(argv[2], OREAD);
if(fd < 0)
    error(1, argv[2]);
d = dirfstat(fd);
if(d == nil)
    error(1, "stat");
n = d->length/sizeof(data[0]);
if(n < 2)
    error(0, "data file too short");
data = malloc(d->length);
if(data == 0)
    error(1, "malloc");
if(read(fd, data, d->length) < 0)
    error(1, "text read");
close(fd);
for(i=0; i<n; i++)
    data[i] = beswal(data[i]);
delta = data[0]-data[1];
print("total: %ld in kernel text: %ld outside kernel text: %ld\n",
    data[0], delta, data[1]);
if(data[0] == 0)
    exits(0);
if (!textsym(&s, 0))
    error(0, "no text symbols");

tbase = mach->kbase;
if(tbase != s.value & ~0xFFF)
    print("warning: kbase %.8llux != tbase %.8llux\n",
        tbase, s.value&~0xFFF);
print("KTZERO %.8llux PGSIZE %dKb\n", tbase, mach->pgsize/1024);
/*
```

```
     * Accumulate counts for each function
     */
    cp = 0;
    k = 0;
    for (i = 0, j = 2; j < n; i++) {
        name = s.name;  /* save name */
        if (!textsym(&s, i)) /* get next symbol */
            break;
        s.value -= tbase;
        s.value /= PCRES;
        sum = 0;
        while (j < n && j < s.value)
            sum += data[j++];
        if (sum) {
            cp = realloc(cp, (k+1)*sizeof(struct COUNTER));
            if (cp == 0)
                error(1, "realloc");
            cp[k].name = name;
            cp[k].time = sum;
            k++;
        }
    }
    if (!k)
        error(0, "no counts");
    cp[k].time = 0;   /* "etext" can take no time */
    /*
     * Sort by time and print
     */
    qsort(cp, k, sizeof(struct COUNTER), compar);
    Binit(&outbuf, 1, OWRITE);
    Bprint(&outbuf, "ms   %% sym\n");
    while(--k>=0)
        Bprint(&outbuf, "%ld\t%3lld.%lld\t%s\n",
                cp[k].time,
                100LL*cp[k].time/delta,
                (1000LL*cp[k].time/delta)%10,
                cp[k].name);
    exits(0);
}
```

⟨misc/kprof.c 16a⟩≡

```
#include <u.h>
#include <libc.h>
#include <bio.h>
#include <mach.h>

⟨constant PCRES 14b⟩

⟨struct COUNTER 14c⟩

⟨function error (misc/kprof.c) 14d⟩

⟨function compar 14e⟩
⟨function main (misc/kprof.c) 14f⟩
```

D.1.3 misc/prof.c

⟨struct Data 16b⟩≡ (23b)

```
struct Data
```

```
{
    ushort down;
    ushort right;
    ulong pc;
    ulong count;
    ulong time;
};
```

⟨*struct* Pc 17a⟩≡ (23b)

```
struct Pc
{
    Pc *next;
    ulong pc;
};
```

⟨*struct* Acc 17b⟩≡ (23b)

```
struct Acc
{
    char *name;
    ulong pc;
    ulong ms;
    ulong calls;
};
```

⟨*global* data 17c⟩≡ (23b)

```
Data* data;
```

⟨*global* acc 17d⟩≡ (23b)

```
Acc* acc;
```

⟨*global* ms 17e⟩≡ (23b)

```
ulong ms;
```

⟨*global* nsym 17f⟩≡ (23b)

```
long nsym;
```

⟨*global* ndata 17g⟩≡ (23b)

```
long ndata;
```

⟨*global* dflag 17h⟩≡ (23b)

```
int dflag;
```

⟨*global* rflag 17i⟩≡ (23b)

```
int rflag;
```

⟨*global* bout 17j⟩≡ (23b)

```
Biobuf bout;
```

⟨*global* tabstop 17k⟩≡ (23b)

```
int tabstop = 4;
```

⟨*global* verbose 17l⟩≡ (23b)

```
int verbose;
```

⟨*function main (misc/prof.c)* 18a⟩≡ (23b)

```
void
main(int argc, char *argv[])
{
    char *s;

    s = getenv("tabstop");
    if(s!=nil && strtol(s,0,0)>0)
        tabstop = strtol(s,0,0);
    ARGBEGIN{
    case 'v':
        verbose = 1;
        break;
    case 'd':
        dflag = 1;
        break;
    case 'r':
        rflag = 1;
        break;
    default:
        fprint(2, "usage: prof [-dr] [8.out] [prof.out]\n");
        exits("usage");
    }ARGEND
    Binit(&bout, 1, OWRITE);
    if(argc > 0)
        syms(argv[0]);
    else
        syms(defaout());
    if(argc > 1)
        datas(argv[1]);
    else
        datas("prof.out");
    if(ndata){
        if(dflag)
            graph(0, data[0].down, 0);
        else
            plot();
    }
    exits(0);
}
```

⟨*function* swapdata 18b⟩≡ (23b)

```
void
swapdata(Data *dp)
{
    dp->down = beswab(dp->down);
    dp->right = beswab(dp->right);
    dp->pc = beswal(dp->pc);
    dp->count = beswal(dp->count);
    dp->time = beswal(dp->time);
}
```

⟨*function* acmp 18c⟩≡ (23b)

```
int
acmp(void *va, void *vb)
{
    Acc *a, *b;
    ulong ua, ub;

    a = va;
```

```
    b = vb;
    ua = a->ms;
    ub = b->ms;

    if(ua > ub)
        return 1;
    if(ua < ub)
        return -1;
    return 0;
}
```

⟨*function* syms 19a⟩≡ (23b)

```
void
syms(char *cout)
{
    Fhdr f;
    int fd;

    if((fd = open(cout, 0)) < 0){
        perror(cout);
        exits("open");
    }
    if (!crackhdr(fd, &f)) {
        fprint(2, "can't read text file header\n");
        exits("read");
    }
    if (f.type == FNONE) {
        fprint(2, "text file not an a.out\n");
        exits("file type");
    }
    if (syminit(fd, &f) < 0) {
        fprint(2, "syminit: %r\n");
        exits("syms");
    }
    close(fd);
}
```

⟨*function* datas 19b⟩≡ (23b)

```
void
datas(char *dout)
{
    int fd;
    Dir *d;
    int i;

    if((fd = open(dout, 0)) < 0){
        perror(dout);
        exits("open");
    }
    d = dirfstat(fd);
    if(d == nil){
        perror(dout);
        exits("stat");
    }
    ndata = d->length/sizeof(data[0]);
    data = malloc(ndata*sizeof(Data));
    if(data == 0){
        fprint(2, "prof: can't malloc data\n");
        exits("data malloc");
    }
```

```
    if(read(fd, data, d->length) != d->length){
        fprint(2, "prof: can't read data file\n");
        exits("data read");
    }
    free(d);
    close(fd);
    for (i = 0; i < ndata; i++)
        swapdata(data+i);
}
```

⟨*function* name 20a⟩≡ (23b)

```
char*
name(ulong pc)
{
    Symbol s;
    static char buf[16];

    if (findsym(pc, CTEXT, &s))
        return(s.name);
    snprint(buf, sizeof(buf), "#%lux", pc);
    return buf;
}
```

⟨*function* graph 20b⟩≡ (23b)

```
void
graph(int ind, ulong i, Pc *pc)
{
    long time, count, prgm;
    Pc lpc;

    if(i >= ndata){
        fprint(2, "prof: index out of range %ld [max %ld]\n", i, ndata);
        return;
    }
    count = data[i].count;
    time = data[i].time;
    prgm = data[i].pc;
    if(time < 0)
        time += data[0].time;
    if(data[i].right != 0xFFFF)
        graph(ind, data[i].right, pc);
    indent(ind);
    if(count == 1)
        Bprint(&bout, "%s:%lud\n", name(prgm), time);
    else
        Bprint(&bout, "%s:%lud/%lud\n", name(prgm), time, count);
    if(data[i].down == 0xFFFF)
        return;
    lpc.next = pc;
    lpc.pc = prgm;
    if(!rflag){
        while(pc){
            if(pc->pc == prgm){
                indent(ind+1);
                Bprint(&bout, "...\n");
                return;
            }
            pc = pc->next;
        }
    }
```

```
    graph(ind+1, data[i].down, &lpc);
}
```

⟨*function* symind 21a⟩≡ (23b)

```
/*
 * assume acc is ordered by increasing text address.
 */
long
symind(ulong pc)
{
    int top, bot, mid;

    bot = 0;
    top = nsym;
    for (mid = (bot+top)/2; mid < top; mid = (bot+top)/2) {
        if (pc < acc[mid].pc)
            top = mid;
        else
        if (mid != nsym-1 && pc >= acc[mid+1].pc)
            bot = mid;
        else
            return mid;
    }
    return -1;
}
```

⟨*function* sum 21b⟩≡ (23b)

```
ulong
sum(ulong i)
{
    long j, dtime, time;
    int k;
    static int indent;

    if(i >= ndata){
        fprint(2, "prof: index out of range %ld [max %ld]\n", i, ndata);
        return 0;
    }
    j = symind(data[i].pc);
    time = data[i].time;
    if(time < 0)
        time += data[0].time;
    if (verbose){
        for(k = 0; k < indent; k++)
            print(" ");
        print("%lud: %ld/%lud", i, data[i].time, data[i].count);
        if (j >= 0)
            print(" %s\n", acc[j].name);
        else
            print(" 0x%lux\n", data[i].pc);
    }
    dtime = 0;
    if(data[i].down != 0xFFFF){
        indent++;
        dtime = sum(data[i].down);
        indent--;
    }
    j = symind(data[i].pc);
    if (j >= 0) {
        acc[j].ms += time - dtime;
```

```
        ms += time - dtime;
        acc[j].calls += data[i].count;
    }
    if(data[i].right == 0xFFFF)
        return time;
    return time + sum(data[i].right);
}
```

⟨*function* plot 22a⟩≡ (23b)

```
void
plot(void)
{
    Symbol s;

    for (nsym = 0; textsym(&s, nsym); nsym++) {
        acc = realloc(acc, (nsym+1)*sizeof(Acc));
        if(acc == 0){
            fprint(2, "prof: malloc fail\n");
            exits("acc malloc");
        }
        acc[nsym].name = s.name;
        acc[nsym].pc = s.value;
        acc[nsym].calls = acc[nsym].ms = 0;
    }
    sum(data[0].down);
    qsort(acc, nsym, sizeof(Acc), acmp);
    Bprint(&bout, "  %%     Time     Calls  Name\n");
    if(ms == 0)
        ms = 1;
    while (--nsym >= 0) {
        if(acc[nsym].calls)
            Bprint(&bout, "%4.1f %8.3f %8lud\t%s\n",
                (100.0*acc[nsym].ms)/ms,
                acc[nsym].ms/1000.0,
                acc[nsym].calls,
                acc[nsym].name);
    }
}
```

⟨*function* indent 22b⟩≡ (23b)

```
void
indent(int ind)
{
    int j;

    j = 2*ind;
    while(j >= tabstop){
        Bwrite(&bout, ".\t", 2);
        j -= tabstop;
    }
    if(j)
        Bwrite(&bout, ".                                ", j);
}
```

⟨*global* trans 22c⟩≡ (23b)

```
char* trans[] =
{
    "386",  "8.out",
    "68020",  "2.out",
    "alpha",  "7.out",
```

```
    "amd64", "6.out",
    "arm",  "5.out",
    "mips",  "v.out",
    "power",  "q.out",
    "sparc",  "k.out",
    "spim",  "0.out",
    0,0
};
```

⟨*function* defaout 23a⟩≡ (23b)

```
char*
defaout(void)
{
    char *p;
    int i;

    p = getenv("objtype");
    if(p)
    for(i=0; trans[i]; i+=2)
        if(strcmp(p, trans[i]) == 0)
            return trans[i+1];
    return trans[1];
}
```

⟨misc/prof.c 23b⟩≡

```
#include <u.h>
#include <libc.h>
#include <bio.h>
#include <mach.h>

typedef struct Data Data;
typedef struct Pc Pc;
typedef struct Acc Acc;

⟨struct Data 16b⟩

⟨struct Pc 17a⟩

⟨struct Acc 17b⟩

⟨global data 17c⟩
⟨global acc 17d⟩
⟨global ms 17e⟩
⟨global nsym 17f⟩
⟨global ndata 17g⟩
⟨global dflag 17h⟩
⟨global rflag 17i⟩
⟨global bout 17j⟩
⟨global tabstop 17k⟩
⟨global verbose 17l⟩

void syms(char*);
void datas(char*);
void graph(int, ulong, Pc*);
void plot(void);
char* name(ulong);
void indent(int);
char* defaout(void);

⟨function main (misc/prof.c) 18a⟩
```

⟨*function* swapdata 18b⟩

⟨*function* acmp 18c⟩

⟨*function* syms 19a⟩

⟨*function* datas 19b⟩

⟨*function* name 20a⟩

⟨*function* graph 20b⟩
⟨*function* symind 21a⟩

⟨*function* sum 21b⟩

⟨*function* plot 22a⟩

⟨*function* indent 22b⟩

⟨*global* trans 22c⟩

⟨*function* defaout 23a⟩

D.1.4 misc/trace.c

⟨*function* NS 24a⟩≡ (37b)
```
#define NS(x) ((vlong)x)
```

⟨*function* US 24b⟩≡ (37b)
```
#define US(x) (NS(x) * 1000ULL)
```

⟨*function* MS 24c⟩≡ (37b)
```
#define MS(x) (US(x) * 1000ULL)
```

⟨*function* S 24d⟩≡ (37b)
```
#define S(x) (MS(x) * 1000ULL)
```

⟨*function* numblocks 24e⟩≡ (37b)
```
#define numblocks(a, b) (((a) + (b) - 1) / (b))
```

⟨*function* roundup 24f⟩≡ (37b)
```
#define roundup(a, b) (numblocks((a), (b)) * (b))
```

⟨*enum* _anon_ 24g⟩≡ (37b)
```
enum {
    OneRound = MS(1)/2LL,
    MilliRound = US(1)/2LL,
};
```

⟨*struct* TEvent 24h⟩≡ (37b)
```
struct TEvent {
    Traceevent;
    vlong etime; /* length of block to draw */
};
```

⟨*struct* Task 25a⟩≡ (37b)

```
struct Task {
    int pid;
    char *name;
    int nevents;
    TEvent *events;
    vlong tstart;
    vlong total;
    vlong runtime;
    vlong runmax;
    vlong runthis;
    long runs;
    ulong tevents[Nevent];
};
```

⟨*enum* _anon_ (misc/trace.c) 25b⟩≡ (37b)

```
enum {
    Nevents = 1024,
    Ncolor = 6,
    K = 1024,
};
```

⟨*global* newwin 25c⟩≡ (37b)

```
int newwin;
```

⟨*global* Width 25d⟩≡ (37b)

```
int Width = 1000;
```

⟨*global* Height 25e⟩≡ (37b)

```
int Height = 100;  // Per task
```

⟨*global* topmargin 25f⟩≡ (37b)

```
int topmargin = 8;
```

⟨*global* bottommargin 25g⟩≡ (37b)

```
int bottommargin = 4;
```

⟨*global* lineht 25h⟩≡ (37b)

```
int lineht = 12;
```

⟨*global* wctlfd 25i⟩≡ (37b)

```
int wctlfd;
```

⟨*global* nevents 25j⟩≡ (37b)

```
int nevents;
```

⟨*global* eventbuf 25k⟩≡ (37b)

```
Traceevent *eventbuf;
```

⟨*global* event 25l⟩≡ (37b)

```
TEvent *event;
```

⟨*global* schedstatename 26a⟩≡ (37b)

```
char *schedstatename[] = {
    [SReady] = "Ready",
    [SRun] = "Run",
    [SDead] = "Dead",
    [SSleep] = "Sleep",
    [SUser] =  "User",

    [SAdmit] = "Admit",
    [SRelease] = "Release",
    [SYield] = "Yield",
    [SSlice] = "Slice",
    [SDeadline] = "Deadline",
    [SExpel] = "Expel",
    [SInts] = "Ints",
    [SInte] = "Inte",
};
```

⟨*struct* scale 26b⟩≡ (37b)

```
struct scale {
    vlong scale;
    vlong bigtics;
    vlong littletics;
    int sleep;
};
```

⟨*global* scales 26c⟩≡ (37b)

```
struct scale scales[] = {
    { US(500), US(100), US(50),    0},
    { US(1000), US(500), US(100),   0},
    { US(2000), US(1000), US(200),   0},
    { US(5000), US(1000), US(500),   0},
    { MS(10),  MS(5),  MS(1),   20},
    { MS(20),  MS(10),  MS(2),   20},
    { MS(50),  MS(10),  MS(5),   20},
    { MS(100), MS(50),  MS(10),   20}, /* starting scaleno */
    { MS(200), MS(100), MS(20),   20},
    { MS(500), MS(100), MS(50),   50},
    { MS(1000), MS(500), MS(100), 100},
    { MS(2000), MS(1000), MS(200), 100},
    { MS(5000), MS(1000), MS(500), 100},
    { S(10),  S(50),  S(1),  100},
    { S(20),  S(10),  S(2),  100},
    { S(50),  S(10),  S(5),  100},
    { S(100),  S(50),  S(10),  100},
    { S(200),  S(100),  S(20),  100},
    { S(500),  S(100),  S(50),  100},
    { S(1000), S(500),  S(100),  100},
};
```

⟨*global verbose (misc/trace.c)* 26d⟩≡ (37b)

```
static int verbose;
```

⟨*global* tasks 26e⟩≡ (37b)

```
Task *tasks;
```

⟨*global* cols 26f⟩≡ (37b)

```
static Image *cols[Ncolor][4];
```

⟨*global* profdev 26g⟩≡ (37b)

```
char*profdev = "/proc/trace";
```

⟨*function* usage 27a⟩≡ (37b)

```
static void
usage(void)
{
    fprint(2, "Usage: %s [-d profdev] [-w] [-v] [-t triggerproc] [processes]\n", argv0);
    exits(nil);
}
```

⟨*function* threadmain 27b⟩≡ (37b)

```
void
threadmain(int argc, char **argv)
{
    int fd, i;
    char fname[80];

    fmtinstall('t', timeconv);
    ARGBEGIN {
    case 'd':
        profdev = EARGF(usage());
        break;
    case 'v':
        verbose = 1;
        break;
    case 'w':
        newwin++;
        break;
    case 't':
        triggerproc = (int)strtol(EARGF(usage()), nil, 0);
        break;
    default:
        usage();
    }
    ARGEND;

    fname[sizeof fname - 1] = 0;
    for(i = 0; i < argc; i++){
        snprint(fname, sizeof fname - 2, "/proc/%s/ctl",
                    argv[i]);
        if((fd = open(fname, OWRITE)) < 0){
            fprint(2, "%s: cannot open %s: %r\n",
                        argv[0], fname);
            continue;
        }

        if(fprint(fd, "trace 1") < 0)
            fprint(2, "%s: cannot enable tracing on %s: %r\n",
                        argv[0], fname);
        close(fd);
    }

    drawtrace();
}
```

⟨*function* mkcol 27c⟩≡ (37b)

```
static void
mkcol(int i, int c0, int c1, int c2)
{
    cols[i][0] = allocimagemix(display, c0, DWhite);
    cols[i][1] = allocimage(display, Rect(0,0,1,1), view->chan, 1, c1);
    cols[i][2] = allocimage(display, Rect(0,0,1,1), view->chan, 1, c2);
```

```
    cols[i][3] = allocimage(display, Rect(0,0,1,1), view->chan, 1, c0);
}
```

⟨*function* colinit 28a⟩≡ (37b)

```
static void
colinit(void)
{
    mediumfont = openfont(display, "/lib/font/bit/lucidasans/unicode.10.font");
    if(mediumfont == nil)
        mediumfont = font;
    tinyfont = openfont(display, "/lib/font/bit/lucidasans/unicode.7.font");
    if(tinyfont == nil)
        tinyfont = font;
    topmargin = mediumfont->height+2;
    bottommargin = tinyfont->height+2;

    /* Peach */
    mkcol(0, 0xFFAAAAFF, 0xFFAAAAFF, 0xBB5D5DFF);
    /* Aqua */
    mkcol(1, DPalebluegreen, DPalegreygreen, DPurpleblue);
    /* Yellow */
    mkcol(2, DPaleyellow, DDarkyellow, DYellowgreen);
    /* Green */
    mkcol(3, DPalegreen, DMedgreen, DDarkgreen);
    /* Blue */
    mkcol(4, 0x00AAFFFF, 0x00AAFFFF, 0x0088CCFF);
    /* Grey */
    cols[5][0] = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xEEEEEEFF);
    cols[5][1] = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xCCCCCCFF);
    cols[5][2] = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0x888888FF);
    cols[5][3] = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xAAAAAAFF);
    grey = cols[5][2];
    red = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xFF0000FF);
    green = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0x00FF00FF);
    blue = allocimage(display, Rect(0,0,1,1), view->chan, 1, 0x0000FFFF);
    bg = display->white;
    fg = display->black;
}
```

⟨*function* time2x 28b⟩≡ (37b)

```
#define time2x(t) ((int)(((t) - oldestts) / ppp))
```

⟨*function* redraw 28c⟩≡ (37b)

```
static void
redraw(int scaleno)
{
    int n, i, j, x;
    char buf[256];
    Point p, q;
    Rectangle r, rtime;
    Task *t;
    vlong ts, oldestts, newestts, period, ppp, scale, s, ss;

    scale = scales[scaleno].scale;
    period = scale + scales[scaleno].littletics;
    ppp = period / Width; // period per pixel.

    /* Round ‘now’ to a nice number */
    newestts = now - (now % scales[scaleno].bigtics) +
            (scales[scaleno].littletics>>1);
```

```
    oldestts = newestts - period;

//print("newestts %t, period %t, %d-%d\n", newestts, period, time2x(oldestts), time2x(newestts));
    if (prevts < oldestts){
        oldestts = newestts - period;

        prevts = oldestts;
        draw(view, view->r, bg, nil, ZP);
    }else{
        /* just white out time */
        rtime = view->r;
        rtime.min.x = rtime.max.x - stringwidth(mediumfont, "00000000000.000s");
        rtime.max.y = rtime.min.y + mediumfont->height;
        draw(view, rtime, bg, nil, ZP);
    }
    p = view->r.min;
    for (n = 0; n != ntasks; n++) {
        t = &tasks[n];
        /* p is upper left corner for this task */
        rtime = Rpt(p, addpt(p, Pt(500, mediumfont->height)));
        draw(view, rtime, bg, nil, ZP);
        snprint(buf, sizeof(buf), "%d %s", t->pid, t->name);
        q = string(view, p, fg, ZP, mediumfont, buf);
        s = now - t->tstart;
        if(t->tevents[SRelease])
            snprint(buf, sizeof(buf), " per %t | avg: %t max: %t",
                (vlong)(s/t->tevents[SRelease]),
                (vlong)(t->runtime/t->tevents[SRelease]),
                t->runmax);
        else if((s /=1000000000LL) != 0)
            snprint(buf, sizeof(buf), " per 1s | avg: %t total: %t",
                t->total/s,
                t->total);
        else
            snprint(buf, sizeof(buf), " total: %t", t->total);
        string(view, q, fg, ZP, tinyfont, buf);
        p.y += Height;
    }
    x = time2x(prevts);

    p = view->r.min;
    for (n = 0; n != ntasks; n++) {
        t = &tasks[n];

        /* p is upper left corner for this task */

        /* Move part already drawn */
        r = Rect(p.x, p.y + topmargin, p.x + x, p.y+Height);
        draw(view, r, view, nil, Pt(p.x + Width - x, p.y + topmargin));

        r.max.x = view->r.max.x;
        r.min.x += x;
        draw(view, r, bg, nil, ZP);

        line(view, addpt(p, Pt(x, Height - lineht)), Pt(view->r.max.x, p.y + Height - lineht),
            Endsquare, Endsquare, 0, cols[n % Ncolor][1], ZP);

        for (i = 0; i < t->nevents-1; i++)
            if (prevts < t->events[i + 1].time)
```

```
        break;

if (i > 0) {
    memmove(t->events, t->events + i, (t->nevents - i) * sizeof(TEvent));
    t->nevents -= i;
}

for (i = 0; i != t->nevents; i++) {
    TEvent *e = &t->events[i], *_e;
    int sx, ex;

    switch (e->etype & 0xffff) {
    case SAdmit:
        if (e->time > prevts && e->time <= newestts) {
            sx = time2x(e->time);
            line(view, addpt(p, Pt(sx, topmargin)),
                addpt(p, Pt(sx, Height - bottommargin)),
                Endarrow, Endsquare, 1, green, ZP);
        }
        break;
    case SExpel:
        if (e->time > prevts && e->time <= newestts) {
            sx = time2x(e->time);
            line(view, addpt(p, Pt(sx, topmargin)),
                addpt(p, Pt(sx, Height - bottommargin)),
                Endsquare, Endarrow, 1, red, ZP);
        }
        break;
    case SRelease:
        if (e->time > prevts && e->time <= newestts) {
            sx = time2x(e->time);
            line(view, addpt(p, Pt(sx, topmargin)),
                addpt(p, Pt(sx, Height - bottommargin)),
                Endarrow, Endsquare, 1, fg, ZP);
        }
        break;
    case SDeadline:
        if (e->time > prevts && e->time <= newestts) {
            sx = time2x(e->time);
            line(view, addpt(p, Pt(sx, topmargin)),
                addpt(p, Pt(sx, Height - bottommargin)),
                Endsquare, Endarrow, 1, fg, ZP);
        }
        break;

    case SYield:
    case SUser:
        if (e->time > prevts && e->time <= newestts) {
            sx = time2x(e->time);
            line(view, addpt(p, Pt(sx, topmargin)),
                addpt(p, Pt(sx, Height - bottommargin)),
                Endsquare, Endarrow, 0,
                (e->etype == SYield)? green: blue, ZP);
        }
        break;
    case SSlice:
        if (e->time > prevts && e->time <= newestts) {
            sx = time2x(e->time);
            line(view, addpt(p, Pt(sx, topmargin)),
                addpt(p, Pt(sx, Height - bottommargin)),
```

```
                Endsquare, Endarrow, 0, red, ZP);
            }
            break;

        case SRun:
            sx = time2x(e->time);
            ex = time2x(e->etime);
            if(ex == sx)
                ex++;

            r = Rect(sx, topmargin + 8, ex, Height - lineht);
            r = rectaddpt(r, p);

            draw(view, r, cols[n % Ncolor][e->etype==SRun?1:3], nil, ZP);

            if(t->pid == triggerproc && ex < Width)
                paused ^= 1;

            for(j = 0; j < t->nevents; j++){
                _e = &t->events[j];
                switch(_e->etype & 0xffff){
                case SInts:
                    if (_e->time > prevts && _e->time <= newestts){
                        sx = time2x(_e->time);
                        line(view, addpt(p, Pt(sx, topmargin)),
                                            addpt(p, Pt(sx, Height / 2 - bottommargin)),
                                            Endsquare, Endsquare, 0,
                                            green, ZP);
                    }
                    break;
                case SInte:
                    if (_e->time > prevts && _e->time <= newestts) {
                        sx = time2x(_e->time);
                        line(view, addpt(p, Pt(sx, Height / 2 - bottommargin)),
                                            addpt(p, Pt(sx, Height - bottommargin)),
                                            Endsquare, Endsquare, 0,
                                            blue, ZP);
                    }
                    break;
                }
            }
            break;
        }
    }
    p.y += Height;
}

ts = prevts + scales[scaleno].littletics - (prevts % scales[scaleno].littletics);
x = time2x(ts);

while(x < Width){
    p = view->r.min;
    for(n = 0; n < ntasks; n++){
        int height, width;

        /* p is upper left corner for this task */
        if ((ts % scales[scaleno].scale) == 0){
            height = 10 * Height;
            width = 1;
        }else if ((ts % scales[scaleno].bigtics) == 0){
```

```
                height = 12 * Height;
                width = 0;
            }else{
                height = 13 * Height;
                width = 0;
            }
            height >>= 4;

            line(view, addpt(p, Pt(x, height)), addpt(p, Pt(x, Height - lineht)),
                Endsquare, Endsquare, width, cols[n % Ncolor][2], ZP);

            p.y += Height;
        }
        ts += scales[scaleno].littletics;
        x = time2x(ts);
    }

    rtime = view->r;
    rtime.min.y = rtime.max.y - tinyfont->height + 2;
    draw(view, rtime, bg, nil, ZP);
    ts = oldestts + scales[scaleno].bigtics - (oldestts % scales[scaleno].bigtics);
    x = time2x(ts);
    ss = 0;
    while(x < Width){
        snprint(buf, sizeof(buf), "%t", ss);
        string(view, addpt(p, Pt(x - stringwidth(tinyfont, buf)/2, - tinyfont->height - 1)),
            fg, ZP, tinyfont, buf);
        ts += scales[scaleno].bigtics;
        ss += scales[scaleno].bigtics;
        x = time2x(ts);
    }

    snprint(buf, sizeof(buf), "%t", now);
    string(view, Pt(view->r.max.x - stringwidth(mediumfont, buf), view->r.min.y),
        fg, ZP, mediumfont, buf);

    flushimage(display, 1);
    prevts = newestts;
}
```

⟨*function* newtask 32⟩≡ (37b)

```
Task*
newtask(ulong pid)
{
    Task *t;
    char buf[64], *p;
    int fd,n;

    tasks = realloc(tasks, (ntasks + 1) * sizeof(Task));
    assert(tasks);

    t = &tasks[ntasks++];
    memset(t, 0, sizeof(Task));
    t->events = nil;
    snprint(buf, sizeof buf, "/proc/%ld/status", pid);
    t->name = nil;
    fd = open(buf, OREAD);
    if (fd >= 0){
        n = read(fd, buf, sizeof buf);
        if(n > 0){
```

```
            p = buf + sizeof buf - 1;
            *p = 0;
            p = strchr(buf, ' ');
            if (p) *p = 0;
            t->name = strdup(buf);
        }else
            print("%s: %r\n", buf);
        close(fd);
    }else
        print("%s: %r\n", buf);
    t->pid = pid;
    prevts = 0;
    if (newwin){
        fprint(wctlfd, "resize -dx %d -dy %d\n",
            Width + 20, (ntasks * Height) + 5);
    }else
        Height = ntasks ? Dy(view->r)/ntasks : Dy(view->r);
    return t;
}
```

⟨*function* doevent 33⟩≡ (37b)

```
void
doevent(Task *t, Traceevent *ep)
{
    int i, n;
    TEvent *event;
    vlong runt;

    t->tevents[ep->etype & 0xffff]++;
    n = t->nevents++;
    t->events = realloc(t->events, t->nevents*sizeof(TEvent));
    assert(t->events);
    event = &t->events[n];
    memmove(event, ep, sizeof(Traceevent));
    event->etime = 0;

    switch(event->etype & 0xffff){
    case SRelease:
        if (t->runthis > t->runmax)
            t->runmax = t->runthis;
        t->runthis = 0;
        break;

    case SSleep:
    case SYield:
    case SReady:
    case SSlice:
        for(i = n-1; i >= 0; i--)
            if (t->events[i].etype == SRun)
                break;
        if(i < 0 || t->events[i].etime != 0)
            break;
        runt = event->time - t->events[i].time;
        if(runt > 0){
            t->events[i].etime = event->time;
            t->runtime += runt;
            t->total += runt;
            t->runthis += runt;
            t->runs++;
        }
```

```
        break;
    case SDead:
print("task died %ld %t %s\n", event->pid, event->time, schedstatename[event->etype & 0xffff]);
        free(t->events);
        free(t->name);
        ntasks--;
        memmove(t, t+1, sizeof(Task)*(&tasks[ntasks]-t));
        if (newwin)
            fprint(wctlfd, "resize -dx %d -dy %d\n",
                Width + 20, (ntasks * Height) + 5);
        else
            Height = ntasks ? Dy(view->r)/ntasks : Dy(view->r);
        prevts = 0;
    }
}
```

⟨*function* drawtrace 34⟩≡ (37b)

```
void
drawtrace(void)
{
    char *wsys, line[256];
    int wfd, logfd;
    Mousectl *mousectl;
    Keyboardctl *keyboardctl;
    int scaleno;
    Rune r;
    int i, n;
    Task *t;
    Traceevent *ep;

    eventbuf = malloc(Nevents*sizeof(Traceevent));
    assert(eventbuf);

    if((logfd = open(profdev, OREAD)) < 0)
        sysfatal("%s: Cannot open %s: %r", argv0, profdev);

    if(newwin){
        if((wsys = getenv("wsys")) == nil)
            sysfatal("%s: Cannot find windowing system: %r",
                        argv0);

        if((wfd = open(wsys, ORDWR)) < 0)
            sysfatal("%s: Cannot open windowing system: %r",
                        argv0);

        snprint(line, sizeof(line), "new -pid %d -dx %d -dy %d",
                getpid(), Width + 20, Height + 5);
        line[sizeof(line) - 1] = '\0';
        rfork(RFNAMEG);

        if(mount(wfd, -1, "/mnt/wsys", MREPL, line) < 0)
            sysfatal("%s: Cannot mount %s under /mnt/wsys: %r",
                        argv0, line);

        if(bind("/mnt/wsys", "/dev", MBEFORE) < 0)
            sysfatal("%s: Cannot bind /mnt/wsys in /dev: %r",
                        argv0);

    }
    if((wctlfd = open("/dev/wctl", OWRITE)) < 0)
```

```
    sysfatal("%s: Cannot open /dev/wctl: %r", argv0);
if(initdraw(nil, nil, "trace") < 0)
    sysfatal("%s: initdraw failure: %r", argv0);

Width = Dx(view->r);
Height = Dy(view->r);

if((mousectl = initmouse(nil, view)) == nil)
    sysfatal("%s: cannot initialize mouse: %r", argv0);

if((keyboardctl = initkeyboard(nil)) == nil)
    sysfatal("%s: cannot initialize keyboard: %r", argv0);

colinit();

paused = 0;
scaleno = 7; /* 100 milliseconds */
now = nsec();
for(;;) {
    Alt a[] = {
        { mousectl->c,   nil,  CHANRCV  },
        { mousectl->resizec, nil,  CHANRCV  },
        { keyboardctl->c,  &r,   CHANRCV  },
        { nil,     nil,  CHANNOBLK },
    };

    switch (alt(a)) {
    case 0:
        continue;

    case 1:
        if(getwindow(display, Refnone) < 0)
            sysfatal("drawrt: Cannot re-attach window");
        if(newwin){
            if(Dx(view->r) != Width ||
                Dy(view->r) != (ntasks * Height)){
                fprint(2, "resize: x: have %d, need %d; y: have %d, need %d\n",
                        Dx(view->r), Width + 8, Dy(view->r), (ntasks * Height) + 8);
                fprint(wctlfd, "resize -dx %d -dy %d\n",
                        Width + 8, (ntasks * Height) + 8);
            }
        }
        else{
            Width = Dx(view->r);
            Height = ntasks? Dy(view->r)/ntasks:
                        Dy(view->r);
        }
        break;

    case 2:

        switch(r){
        case 'r':
            for(i = 0; i < ntasks; i++){
                tasks[i].tstart = now;
                tasks[i].total = 0;
                tasks[i].runtime = 0;
                tasks[i].runmax = 0;
                tasks[i].runthis = 0;
                tasks[i].runs = 0;
```

```
            memset(tasks[i].tevents, 0, Nevent*sizeof(ulong));

        }
        break;

    case 'p':
        paused ^= 1;
        prevts = 0;
        break;

    case '-':
        if (scaleno < nelem(scales) - 1)
            scaleno++;
        prevts = 0;
        break;

    case '+':
        if (scaleno > 0)
            scaleno--;
        prevts = 0;
        break;

    case 'q':
        threadexitsall(nil);

    case 'v':
        verbose ^= 1;

    default:
        break;
    }
    break;

case 3:
    now = nsec();
    while((n = read(logfd, eventbuf, Nevents*sizeof(Traceevent))) > 0){
        assert((n % sizeof(Traceevent)) == 0);
        nevents = n / sizeof(Traceevent);
        for (ep = eventbuf; ep < eventbuf + nevents; ep++){
            if ((ep->etype & 0xffff) >= Nevent){
                print("%ld %t Illegal event %ld\n",
                    ep->pid, ep->time, ep->etype & 0xffff);
                continue;
            }
            if (verbose)
                print("%ld %t %s\n",
                    ep->pid, ep->time, schedstatename[ep->etype & 0xffff]);

            for(i = 0; i < ntasks; i++)
                if(tasks[i].pid == ep->pid)
                    break;

            if(i == ntasks){
                t = newtask(ep->pid);
                t->tstart = ep->time;
            }else
                t = &tasks[i];

            doevent(t, ep);
        }
```

```
            }
            if(!paused)
                redraw(scaleno);
        }
        sleep(scales[scaleno].sleep);
    }
}
```

⟨*function* timeconv 37a⟩≡ (37b)

```
int
timeconv(Fmt *f)
{
    char buf[128], *sign;
    vlong t;

    buf[0] = 0;
    switch(f->r) {
    case 'U':
        t = va_arg(f->args, vlong);
        break;
    case 't':  // vlong in nanoseconds
        t = va_arg(f->args, vlong);
        break;
    default:
        return fmtstrcpy(f, "(timeconv)");
    }
    if (t < 0) {
        sign = "-";
        t = -t;
    }else
        sign = "";
    if (t > S(1)){
        t += OneRound;
        sprint(buf, "%s%d.%.3ds", sign, (int)(t / S(1)), (int)(t % S(1))/1000000);
    }else if (t > MS(1)){
        t += MilliRound;
        sprint(buf, "%s%d.%.3dms", sign, (int)(t / MS(1)), (int)(t % MS(1))/1000);
    }else if (t > US(1))
        sprint(buf, "%s%d.%.3dµs", sign, (int)(t / US(1)), (int)(t % US(1)));
    else
        sprint(buf, "%s%dns", sign, (int)t);
    return fmtstrcpy(f, buf);
}
```

⟨misc/trace.c 37b⟩≡

```
#include <u.h>
#include <tos.h>
#include <libc.h>
#include <thread.h>
#include <ip.h>
#include <bio.h>

#include <draw.h>
#include <window.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>

#include <trace.h>
```

```
// a GUI tracer for kernel scheduler events

#pragma varargck type "t"  vlong
#pragma varargck type "U"  uvlong

⟨function NS 24a⟩
⟨function US 24b⟩
⟨function MS 24c⟩
⟨function S 24d⟩

⟨function numblocks 24e⟩
⟨function roundup 24f⟩

⟨enum _anon_ 24g⟩

typedef struct TEvent TEvent;
typedef struct Task Task;
⟨struct TEvent 24h⟩

⟨struct Task 25a⟩

⟨enum _anon_ (misc/trace.c) 25b⟩

vlong now, prevts;

⟨global newwin 25c⟩
⟨global Width 25d⟩
⟨global Height 25e⟩
⟨global topmargin 25f⟩
⟨global bottommargin 25g⟩
⟨global lineht 25h⟩
⟨global wctlfd 25i⟩
⟨global nevents 25j⟩
⟨global eventbuf 25k⟩
⟨global event 25l⟩

void drawtrace(void);
int schedparse(char*, char*, char*);
int timeconv(Fmt*);

⟨global schedstatename 26a⟩

⟨struct scale 26b⟩

⟨global scales 26c⟩

int ntasks, triggerproc, paused;
⟨global verbose (misc/trace.c) 26d⟩
⟨global tasks 26e⟩
⟨global cols 26f⟩
static Font *mediumfont, *tinyfont;
Image *grey, *red, *green, *blue, *bg, *fg;
⟨global profdev 26g⟩

⟨function usage 27a⟩

⟨function threadmain 27b⟩

⟨function mkcol 27c⟩
```

⟨*function* colinit 28a⟩

⟨*function* time2x 28b⟩

⟨*function* redraw 28c⟩

⟨*function* newtask 32⟩

⟨*function* doevent 33⟩

⟨*function* drawtrace 34⟩

⟨*function* timeconv 37a⟩

D.1.5 misc/tprof.c

⟨*constant PCRES (misc/tprof.c)* 39a⟩≡ (41)

```
#define PCRES 8
```

⟨*struct COUNTER (misc/tprof.c)* 39b⟩≡ (41)

```
struct COUNTER
{
    char  *name;  /* function name */
    long time;  /* ticks spent there */
};
```

⟨*function error (misc/tprof.c)* 39c⟩≡ (41)

```
static void
error(int perr, char *s)
{
    fprint(2, "tprof: %s", s);
    if(perr){
        fprint(2, ": ");
        perror(0);
    }else
        fprint(2, "\n");
    exits(s);
}
```

⟨*function compar (misc/tprof.c)* 39d⟩≡ (41)

```
int
compar(void *va, void *vb)
{
    struct COUNTER *a, *b;

    a = va;
    b = vb;
    if(a->time < b->time)
        return -1;
    if(a->time == b->time)
        return 0;
    return 1;
}
```

⟨*function main (misc/tprof.c)* 39e⟩≡ (41)

```
void
main(int argc, char *argv[])
{
    int fd;
```

```
long i, j, k, n;
Dir *d;
char *name;
ulong *data;
ulong tbase, sum;
long delta;
Symbol s;
Biobuf outbuf;
Fhdr f;
struct COUNTER *cp;
char filebuf[128], *file;

if(argc != 2 && argc != 3)
    error(0, "usage: tprof pid [binary]");
/*
 * Read symbol table
 */
if(argc == 2){
    file = filebuf;
    snprint(filebuf, sizeof filebuf, "/proc/%s/text", argv[1]);
}else
    file = argv[2];

fd = open(file, OREAD);
if(fd < 0)
    error(1, file);

if (!crackhdr(fd, &f))
    error(1, "read text header");
if (f.type == FNONE)
    error(0, "text file not an a.out");
machbytype(f.type);
if (syminit(fd, &f) < 0)
    error(1, "syminit");
close(fd);
/*
 * Read timing data
 */
file = smprint("/proc/%s/profile", argv[1]);
fd = open(file, OREAD);
if(fd < 0)
    error(1, file);
free(file);
d = dirfstat(fd);
if(d == nil)
    error(1, "stat");
n = d->length/sizeof(data[0]);
if(n < 2)
    error(0, "data file too short");
data = malloc(d->length);
if(data == 0)
    error(1, "malloc");
if(read(fd, data, d->length) < 0)
    error(1, "text read");
close(fd);

for(i=0; i<n; i++)
    data[i] = machdata->swal(data[i]);

delta = data[0]-data[1];
```

```
    print("total: %ld\n", data[0]);
    if(data[0] == 0)
        exits(0);
    if (!textsym(&s, 0))
        error(0, "no text symbols");
    tbase = s.value & ~(mach->pgsize-1); /* align down to page */
    print("TEXT %.8lux\n", tbase);
    /*
     * Accumulate counts for each function
     */
    cp = 0;
    k = 0;
    for (i = 0, j = (s.value-tbase)/PCRES+2; j < n; i++) {
        name = s.name;  /* save name */
        if (!textsym(&s, i)) /* get next symbol */
            break;
        sum = 0;
        while (j < n && j*PCRES < s.value-tbase)
            sum += data[j++];
        if (sum) {
            cp = realloc(cp, (k+1)*sizeof(struct COUNTER));
            if (cp == 0)
                error(1, "realloc");
            cp[k].name = name;
            cp[k].time = sum;
            k++;
        }
    }
    if (!k)
        error(0, "no counts");
    cp[k].time = 0;   /* "etext" can take no time */
    /*
     * Sort by time and print
     */
    qsort(cp, k, sizeof(struct COUNTER), compar);
    Binit(&outbuf, 1, OWRITE);
    Bprint(&outbuf, "    ms      %%   sym\n");
    while(--k>=0)
        Bprint(&outbuf, "%6ld\t%3lld.%lld\t%s\n",
                cp[k].time,
                100LL*cp[k].time/delta,
                (1000LL*cp[k].time/delta)%10,
                cp[k].name);
    exits(0);
}
```

⟨misc/tprof.c 41⟩≡

```
#include <u.h>
#include <libc.h>
#include <bio.h>
#include <mach.h>
```

⟨*constant PCRES (misc/tprof.c)* 39a⟩

⟨*struct COUNTER (misc/tprof.c)* 39b⟩

⟨*function error (misc/tprof.c)* 39c⟩

⟨*function compar (misc/tprof.c)* 39d⟩
⟨*function main (misc/tprof.c)* 39e⟩

D.1.6 misc/stats.c

⟨*constant* MAXNUM 42a⟩≡ (67)

```
// a GUI system monitoring tool

#define MAXNUM 10 /* maximum number of numbers on data line */
```

⟨*struct* Graph 42b⟩≡ (67)

```
struct Graph
{
    int  colindex;
    Rectangle r;
    uvlong  *data;
    int  ndata;
    char  *label;
    void  (*newvalue)(Machine*, uvlong*, uvlong*, int);
    void  (*update)(Graph*, uvlong, uvlong);
    Machine  *mach;
    int  overflow;
    Image  *overtmp;
};
```

⟨*enum* _anon_ (misc/stats.c) 42c⟩≡ (67)

```
enum
{
    /* old /dev/swap */
    Mem  = 0,
    Maxmem,
    Swap,
    Maxswap,

    /* /dev/sysstats */
    Procno = 0,
    Context,
    Interrupt,
    Syscall,
    Fault,
    TLBfault,
    TLBpurge,
    Load,
    Idle,
    InIntr,
    /* /net/ether0/stats */
    In  = 0,
    Link,
    Out,
    Err0,
};
```

⟨*struct* Machine 42d⟩≡ (67)

```
struct Machine
{
    char  *name;
    char  *shortname;
    int  remote;
    int  statsfd;
    int  swapfd;
    int  etherfd;
    int  ifstatsfd;
    int  batteryfd;
```

```
    int  bitsybatfd;
    int  tempfd;
    int  disable;

    uvlong  devswap[4];
    uvlong  devsysstat[10];
    uvlong  prevsysstat[10];
    int  nproc;
    int  lgproc;
    uvlong  netetherstats[8];
    uvlong  prevetherstats[8];
    uvlong  batterystats[2];
    uvlong  netetherifstats[2];
    uvlong  temp[10];

    /* big enough to hold /dev/sysstat even with many processors */
    char  buf[8*1024];
    char  *bufp;
    char  *ebufp;
};
```

⟨*enum* _anon_ (misc/stats.c)2 43a⟩≡ (67)

```
enum
{
    Mainproc,
    Mouseproc,
    NPROC,
};
```

⟨*enum* _anon_ (misc/stats.c)3 43b⟩≡ (67)

```
enum
{
    Ncolor  = 6,
    Ysqueeze = 2, /* vertical squeezing of label text */
    Labspace = 2, /* room around label */
    Dot  = 2, /* height of dot */
    Opwid  = 5, /* strlen("add  ") or strlen("drop ") */
    Nlab  = 3, /* max number of labels on y axis */
    Lablen  = 16, /* max length of label */
    Lx  = 4, /* label tick length */
};
```

⟨*enum* Menu2 43c⟩≡ (67)

```
enum Menu2
{
    Mbattery,
    Mcontext,
    Mether,
    Methererr,
    Metherin,
    Metherout,
    Mfault,
    Midle,
    Minintr,
    Mintr,
    Mload,
    Mmem,
    Mswap,
    Msyscall,
    Mtlbmiss,
```

```
    Mtlbpurge,
    Msignal,
    Mtemp,
    Nmenu2,
};
```

⟨*global* menu2str 44a⟩≡ (67)

```
char *menu2str[Nmenu2+1] = {
    "add  battery ",
    "add  context ",
    "add  ether   ",
    "add  ethererr",
    "add  etherin ",
    "add  etherout",
    "add  fault   ",
    "add  idle    ",
    "add  inintr  ",
    "add  intr    ",
    "add  load    ",
    "add  mem     ",
    "add  swap    ",
    "add  syscall ",
    "add  tlbmiss ",
    "add  tlbpurge",
    "add  802.11b ",
    "add  temp    ",
    nil,
};
```

⟨*global* menu2 44b⟩≡ (67)

```
Menu menu2 = {menu2str, nil};
```

⟨*global* present 44c⟩≡ (67)

```
int present[Nmenu2];
```

⟨*global* newvaluefn 44d⟩≡ (67)

```
void (*newvaluefn[Nmenu2])(Machine*, uvlong*, uvlong*, int init) = {
    batteryval,
    contextval,
    etherval,
    ethererrval,
    etherinval,
    etheroutval,
    faultval,
    idleval,
    inintrval,
    intrval,
    loadval,
    memval,
    swapval,
    syscallval,
    tlbmissval,
    tlbpurgeval,
    signalval,
    tempval,
};
```

⟨*global cols (misc/stats.c)* 44e⟩≡ (67)

```
Image *cols[Ncolor][3];
```

⟨*global* graph 45a⟩≡ (67)
```
Graph *graph;
```

⟨*global* mach 45b⟩≡ (67)
```
Machine *mach;
```

⟨*global* mediumfont 45c⟩≡ (67)
```
Font *mediumfont;
```

⟨*global* mysysname 45d⟩≡ (67)
```
char *mysysname;
```

⟨*global* argchars 45e⟩≡ (67)
```
char argchars[] = "8bceEfiImlnpstwz";
```

⟨*global* pids 45f⟩≡ (67)
```
int pids[NPROC];
```

⟨*global* parity 45g⟩≡ (67)
```
int  parity; /* toggled to avoid patterns in textured background */
```

⟨*global* nmach 45h⟩≡ (67)
```
int nmach;
```

⟨*global* ngraph 45i⟩≡ (67)
```
int ngraph; /* totaly number is ngraph*nmach */
```

⟨*global* scale 45j⟩≡ (67)
```
double scale = 1.0;
```

⟨*global* logscale 45k⟩≡ (67)
```
int logscale = 0;
```

⟨*global* ylabels 45l⟩≡ (67)
```
int ylabels = 0;
```

⟨*global* oldsystem 45m⟩≡ (67)
```
int oldsystem = 0;
```

⟨*global* sleeptime 45n⟩≡ (67)
```
int  sleeptime = 1000;
```

⟨*global* procnames 45o⟩≡ (67)
```
char *procnames[NPROC] = {"main", "mouse"};
```

⟨*function* killall 45p⟩≡ (67)
```
void
killall(char *s)
{
    int i, pid;

    pid = getpid();
    for(i=0; i<NPROC; i++)
        if(pids[i] && pids[i]!=pid)
            postnote(PNPROC, pids[i], "kill");
    exits(s);
}
```

⟨*function* emalloc 46a⟩≡ (67)

```
void*
emalloc(ulong sz)
{
    void *v;
    v = malloc(sz);
    if(v == nil) {
        fprint(2, "stats: out of memory allocating %ld: %r\n", sz);
        killall("mem");
    }
    memset(v, 0, sz);
    return v;
}
```

⟨*function* erealloc 46b⟩≡ (67)

```
void*
erealloc(void *v, ulong sz)
{
    v = realloc(v, sz);
    if(v == nil) {
        fprint(2, "stats: out of memory reallocating %ld: %r\n", sz);
        killall("mem");
    }
    return v;
}
```

⟨*function* estrdup 46c⟩≡ (67)

```
char*
estrdup(char *s)
{
    char *t;
    if((t = strdup(s)) == nil) {
        fprint(2, "stats: out of memory in strdup(%.10s): %r\n", s);
        killall("mem");
    }
    return t;
}
```

⟨*function mkcol (misc/stats.c)* 46d⟩≡ (67)

```
void
mkcol(int i, int c0, int c1, int c2)
{
    cols[i][0] = allocimagemix(display, c0, DWhite);
    cols[i][1] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, c1);
    cols[i][2] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, c2);
}
```

⟨*function colinit (misc/stats.c)* 46e⟩≡ (67)

```
void
colinit(void)
{
    mediumfont = openfont(display, "/lib/font/bit/pelm/latin1.8.font");
    if(mediumfont == nil)
        mediumfont = font;

    /* Peach */
    mkcol(0, 0xFFAAAAFF, 0xFFAAAAFF, 0xBB5D5DFF);
    /* Aqua */
    mkcol(1, DPalebluegreen, DPalegreygreen, DPurpleblue);
    /* Yellow */
```

```
    mkcol(2, DPaleyellow, DDarkyellow, DYellowgreen);
    /* Green */
    mkcol(3, DPalegreen, DMedgreen, DDarkgreen);
    /* Blue */
    mkcol(4, 0x00AAFFFF, 0x00AAFFFF, 0x0088CCFF);
    /* Grey */
    cols[5][0] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, 0xEEEEEEFF);
    cols[5][1] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, 0xCCCCCCFF);
    cols[5][2] = allocimage(display, Rect(0,0,1,1), CMAP8, 1, 0x888888FF);
}
```

⟨*function* loadbuf 47a⟩≡ (67)

```
int
loadbuf(Machine *m, int *fd)
{
    int n;


    if(*fd < 0)
        return 0;
    seek(*fd, 0, 0);
    n = read(*fd, m->buf, sizeof m->buf-1);
    if(n <= 0){
        close(*fd);
        *fd = -1;
        return 0;
    }
    m->bufp = m->buf;
    m->ebufp = m->buf+n;
    m->buf[n] = 0;
    return 1;
}
```

⟨*function* label 47b⟩≡ (67)

```
void
label(Point p, int dy, char *text)
{
    char *s;
    Rune r[2];
    int w, maxw, maxy;

    p.x += Labspace;
    maxy = p.y+dy;
    maxw = 0;
    r[1] = '\0';
    for(s=text; *s; ){
        if(p.y+mediumfont->height-Ysqueeze > maxy)
            break;
        w = chartorune(r, s);
        s += w;
        w = runestringwidth(mediumfont, r);
        if(w > maxw)
            maxw = w;
        runestring(view, p, display->black, ZP, mediumfont, r);
        p.y += mediumfont->height-Ysqueeze;
    }
}
```

⟨*function* paritypt 47c⟩≡ (67)

```
Point
```

```
paritypt(int x)
{
    return Pt(x+parity, 0);
}
```

⟨*function* datapoint 48a⟩≡ (67)

```
Point
datapoint(Graph *g, int x, uvlong v, uvlong vmax)
{
    Point p;
    double y;

    p.x = x;
    y = ((double)v)/(vmax*scale);
    if(logscale){
        /*
         * Arrange scale to cover a factor of 1000.
         * vmax corresponds to the 100 mark.
         * 10*vmax is the top of the scale.
         */
        if(y <= 0.)
            y = 0;
        else{
            y = log10(y);
            /* 1 now corresponds to the top; -2 to the bottom; rescale */
            y = (y+2.)/3.;
        }
    }
    if(y < 0x7fffffff){ /* avoid floating overflow */
        p.y = g->r.max.y - Dy(g->r)*y - Dot;
        if(p.y < g->r.min.y)
            p.y = g->r.min.y;
        if(p.y > g->r.max.y-Dot)
            p.y = g->r.max.y-Dot;
    }else
        p.y = g->r.max.y-Dot;
    return p;
}
```

⟨*function* drawdatum 48b⟩≡ (67)

```
void
drawdatum(Graph *g, int x, uvlong prev, uvlong v, uvlong vmax)
{
    int c;
    Point p, q;

    c = g->colindex;
    p = datapoint(g, x, v, vmax);
    q = datapoint(g, x, prev, vmax);
    if(p.y < q.y){
        draw(view, Rect(p.x, g->r.min.y, p.x+1, p.y), cols[c][0], nil, paritypt(p.x));
        draw(view, Rect(p.x, p.y, p.x+1, q.y+Dot), cols[c][2], nil, ZP);
        draw(view, Rect(p.x, q.y+Dot, p.x+1, g->r.max.y), cols[c][1], nil, ZP);
    }else{
        draw(view, Rect(p.x, g->r.min.y, p.x+1, q.y), cols[c][0], nil, paritypt(p.x));
        draw(view, Rect(p.x, q.y, p.x+1, p.y+Dot), cols[c][2], nil, ZP);
        draw(view, Rect(p.x, p.y+Dot, p.x+1, g->r.max.y), cols[c][1], nil, ZP);
    }

}
```

⟨*function redraw (misc/stats.c)* 49a⟩≡ (67)

```
void
redraw(Graph *g, uvlong vmax)
{
    int i, c;

    c = g->colindex;
    draw(view, g->r, cols[c][0], nil, paritypt(g->r.min.x));
    for(i=1; i<Dx(g->r); i++)
        drawdatum(g, g->r.max.x-i, g->data[i-1], g->data[i], vmax);
    drawdatum(g, g->r.min.x, g->data[i], g->data[i], vmax);
    g->overflow = 0;
}
```

⟨*function* update1 49b⟩≡ (67)

```
void
update1(Graph *g, uvlong v, uvlong vmax)
{
    char buf[48];
    int overflow;

    if(g->overflow && g->overtmp!=nil)
        draw(view, g->overtmp->r, g->overtmp, nil, g->overtmp->r.min);
    draw(view, g->r, view, nil, Pt(g->r.min.x+1, g->r.min.y));
    drawdatum(g, g->r.max.x-1, g->data[0], v, vmax);
    memmove(g->data+1, g->data, (g->ndata-1)*sizeof(g->data[0]));
    g->data[0] = v;
    g->overflow = 0;
    if(logscale)
        overflow = (v>10*vmax*scale);
    else
        overflow = (v>vmax*scale);
    if(overflow && g->overtmp!=nil){
        g->overflow = 1;
        draw(g->overtmp, g->overtmp->r, view, nil, g->overtmp->r.min);
        sprint(buf, "%llud", v);
        string(view, g->overtmp->r.min, display->black, ZP, mediumfont, buf);
    }
}
```

⟨*function* readnums 49c⟩≡ (67)

```
/* read one line of text from buffer and process integers */
int
readnums(Machine *m, int n, uvlong *a, int spanlines)
{
    int i;
    char *p, *q, *ep;

    if(spanlines)
        ep = m->ebufp;
    else
        for(ep=m->bufp; ep<m->ebufp; ep++)
            if(*ep == '\n')
                break;
    p = m->bufp;
    for(i=0; i<n && p<ep; i++){
        while(p<ep && (!isascii(*p) || !isdigit(*p)) && *p!='-')
            p++;
        if(p == ep)
            break;
```

```
        a[i] = strtoull(p, &q, 10);
        p = q;
    }
    if(ep < m->ebufp)
        ep++;
    m->bufp = ep;
    return i == n;
}
```

⟨*function* filter 50a⟩≡ (67)

```
/* Network on fd1, mount driver on fd0 */
static int
filter(int fd)
{
    int p[2];

    if(pipe(p) < 0){
        fprint(2, "stats: can't pipe: %r\n");
        killall("pipe");
    }

    switch(rfork(RFNOWAIT|RFPROC|RFFDG)) {
    case -1:
        sysfatal("rfork record module");
    case 0:
        dup(fd, 1);
        close(fd);
        dup(p[0], 0);
        close(p[0]);
        close(p[1]);
        execl("/bin/aux/fcall", "fcall", nil);
        fprint(2, "stats: can't exec fcall: %r\n");
        killall("fcall");
    default:
        close(fd);
        close(p[0]);
    }
    return p[1];
}
```

⟨*function* connect9fs 50b⟩≡ (67)

```
/*
 * 9fs
 */
int
connect9fs(char *addr)
{
    char dir[256], *na;
    int fd;

    fprint(2, "connect9fs...");
    na = netmkaddr(addr, 0, "9fs");

    fprint(2, "dial %s...", na);
    if((fd = dial(na, 0, dir, 0)) < 0)
        return -1;

    fprint(2, "dir %s...", dir);
// if(strstr(dir, "tcp"))
//  fd = filter(fd);
```

```
    return fd;
}
```

⟨*function* old9p 51a⟩≡ (67)

```
int
old9p(int fd)
{
    int p[2];

    if(pipe(p) < 0)
        return -1;

    switch(rfork(RFPROC|RFFDG|RFNAMEG)) {
    case -1:
        return -1;
    case 0:
        if(fd != 1){
            dup(fd, 1);
            close(fd);
        }
        if(p[0] != 0){
            dup(p[0], 0);
            close(p[0]);
        }
        close(p[1]);
        if(0){
            fd = open("/sys/log/cpu", OWRITE);
            if(fd != 2){
                dup(fd, 2);
                close(fd);
            }
            execl("/bin/srvold9p", "srvold9p", "-ds", nil);
        } else
            execl("/bin/srvold9p", "srvold9p", "-s", nil);
        return -1;
    default:
        close(fd);
        close(p[0]);
    }
    return p[1];
}
```

⟨*function* connectexportfs 51b⟩≡ (67)

```
/*
 * exportfs
 */
int
connectexportfs(char *addr)
{
    char buf[ERRMAX], dir[256], *na;
    int fd, n;
    char *tree;
    AuthInfo *ai;

    tree = "/";
    na = netmkaddr(addr, 0, "exportfs");
    if((fd = dial(na, 0, dir, 0)) < 0)
        return -1;

    ai = auth_proxy(fd, auth_getkey, "proto=p9any role=client");
```

```
    if(ai == nil)
        return -1;

    n = write(fd, tree, strlen(tree));
    if(n < 0){
        close(fd);
        return -1;
    }

    strcpy(buf, "can't read tree");
    n = read(fd, buf, sizeof buf - 1);
    if(n!=2 || buf[0]!='O' || buf[1]!='K'){
        buf[sizeof buf - 1] = '\0';
        werrstr("bad remote tree: %s\n", buf);
        close(fd);
        return -1;
    }

// if(strstr(dir, "tcp"))
//  fd = filter(fd);

    if(oldsystem)
        return old9p(fd);

    return fd;
}
```

⟨*function* readswap 52a⟩≡ (67)

```
int
readswap(Machine *m, uvlong *a)
{
    if(strstr(m->buf, "memory\n")){
        /* new /dev/swap - skip first 3 numbers */
        if(!readnums(m, 7, a, 1))
            return 0;
        a[0] = a[3];
        a[1] = a[4];
        a[2] = a[5];
        a[3] = a[6];
        return 1;
    }
    return readnums(m, nelem(m->devswap), a, 0);
}
```

⟨*function* shortname 52b⟩≡ (67)

```
char*
shortname(char *s)
{
    char *p, *e;

    p = estrdup(s);
    e = strchr(p, '.');
    if(e)
        *e = 0;
    return p;
}
```

⟨*function* ilog10 52c⟩≡ (67)

```
int
ilog10(uvlong j)
```

```
{
    int i;

    for(i = 0; j >= 10; i++)
        j /= 10;
    return i;
}
```

⟨*function* initmach 53⟩≡ (67)

```
int
initmach(Machine *m, char *name)
{
    int n, fd;
    uvlong a[MAXNUM];
    char *p, mpt[256], buf[256];

    p = strchr(name, '!');
    if(p)
        p++;
    else
        p = name;
    m->name = estrdup(p);
    m->shortname = shortname(p);
    m->remote = (strcmp(p, mysysname) != 0);
    if(m->remote == 0)
        strcpy(mpt, "");
    else{
        snprint(mpt, sizeof mpt, "/n/%s", p);
        fd = connectexportfs(name);
        if(fd < 0){
            fprint(2, "can't connect to %s: %r\n", name);
            return 0;
        }
        /* BUG? need to use amount() now? */
        if(mount(fd, -1, mpt, MREPL, "") < 0){
            fprint(2, "stats: mount %s on %s failed (%r); trying /n/sid\n", name, mpt);
            strcpy(mpt, "/n/sid");
            if(mount(fd, -1, mpt, MREPL, "") < 0){
                fprint(2, "stats: mount %s on %s failed: %r\n", name, mpt);
                return 0;
            }
        }
    }

    snprint(buf, sizeof buf, "%s/dev/swap", mpt);
    m->swapfd = open(buf, OREAD);
    if(loadbuf(m, &m->swapfd) && readswap(m, a))
        memmove(m->devswap, a, sizeof m->devswap);
    else{
        m->devswap[Maxswap] = 100;
        m->devswap[Maxmem] = 100;
    }

    snprint(buf, sizeof buf, "%s/dev/sysstat", mpt);
    m->statsfd = open(buf, OREAD);
    if(loadbuf(m, &m->statsfd)){
        for(n=0; readnums(m, nelem(m->devsysstat), a, 0); n++)
            ;
        m->nproc = n;
    }else
```

```
        m->nproc = 1;
    m->lgproc = ilog10(m->nproc);

    snprint(buf, sizeof buf, "%s/net/ether0/stats", mpt);
    m->etherfd = open(buf, OREAD);
    if(loadbuf(m, &m->etherfd) && readnums(m, nelem(m->netetherstats), a, 1))
        memmove(m->netetherstats, a, sizeof m->netetherstats);

    snprint(buf, sizeof buf, "%s/net/ether0/ifstats", mpt);
    m->ifstatsfd = open(buf, OREAD);
    if(loadbuf(m, &m->ifstatsfd)){
        /* need to check that this is a wavelan interface */
        if(strncmp(m->buf, "Signal: ", 8) == 0 && readnums(m, nelem(m->netetherifstats), a, 1))
            memmove(m->netetherifstats, a, sizeof m->netetherifstats);
    }

    snprint(buf, sizeof buf, "%s/mnt/apm/battery", mpt);
    m->batteryfd = open(buf, OREAD);
    m->bitsybatfd = -1;
    if(m->batteryfd >= 0){
        if(loadbuf(m, &m->batteryfd) && readnums(m, nelem(m->batterystats), a, 0))
            memmove(m->batterystats, a, sizeof(m->batterystats));
    }else{
        snprint(buf, sizeof buf, "%s/dev/battery", mpt);
        m->bitsybatfd = open(buf, OREAD);
        if(loadbuf(m, &m->bitsybatfd) && readnums(m, 1, a, 0))
            memmove(m->batterystats, a, sizeof(m->batterystats));
    }
    snprint(buf, sizeof buf, "%s/dev/cputemp", mpt);
    m->tempfd = open(buf, OREAD);
    if(loadbuf(m, &m->tempfd))
        for(n=0; n < nelem(m->temp) && readnums(m, 2, a, 0); n++)
             m->temp[n] = a[0];
    return 1;
}
```

⟨*global* catchalarm 54a⟩≡ (67)

```
jmp_buf catchalarm;
```

⟨*function* alarmed 54b⟩≡ (67)

```
void
alarmed(void *a, char *s)
{
    if(strcmp(s, "alarm") == 0)
        notejmp(a, catchalarm, 1);
    noted(NDFLT);
}
```

⟨*function* needswap 54c⟩≡ (67)

```
int
needswap(int init)
{
    return init | present[Mmem] | present[Mswap];
}
```

⟨*function* needstat 54d⟩≡ (67)

```
int
needstat(int init)
{
    return init | present[Mcontext]  | present[Mfault] | present[Mintr] | present[Mload] | present[Midle] |
        present[Minintr] | present[Msyscall] | present[Mtlbmiss] | present[Mtlbpurge];
}
```

⟨*function* needether 55a⟩≡ (67)

```
int
needether(int init)
{
    return init | present[Mether] | present[Metherin] | present[Metherout] | present[Methererr];
}
```

⟨*function* needbattery 55b⟩≡ (67)

```
int
needbattery(int init)
{
    return init | present[Mbattery];
}
```

⟨*function* needsignal 55c⟩≡ (67)

```
int
needsignal(int init)
{
    return init | present[Msignal];
}
```

⟨*function* needtemp 55d⟩≡ (67)

```
int
needtemp(int init)
{
    return init | present[Mtemp];
}
```

⟨*function* readmach 55e⟩≡ (67)

```
void
readmach(Machine *m, int init)
{
    int n, i;
    uvlong a[nelem(m->devsysstat)];
    char buf[32];

    if(m->remote && (m->disable || setjmp(catchalarm))){
        if (m->disable++ >= 5)
            m->disable = 0; /* give it another chance */
        memmove(m->devsysstat, m->prevsysstat, sizeof m->devsysstat);
        memmove(m->netetherstats, m->prevetherstats, sizeof m->netetherstats);
        return;
    }
    snprint(buf, sizeof buf, "%s", m->name);
    if (strcmp(m->name, buf) != 0){
        free(m->name);
        m->name = estrdup(buf);
        free(m->shortname);
        m->shortname = shortname(buf);
        if(display != nil) /* else we're still initializing */
            eresized(0);
    }
    if(m->remote){
        notify(alarmed);
        alarm(5000);
    }
    if(needswap(init) && loadbuf(m, &m->swapfd) && readswap(m, a))
        memmove(m->devswap, a, sizeof m->devswap);
    if(needstat(init) && loadbuf(m, &m->statsfd)){
        memmove(m->prevsysstat, m->devsysstat, sizeof m->devsysstat);
```

```
        memset(m->devsysstat, 0, sizeof m->devsysstat);
        for(n=0; n<m->nproc && readnums(m, nelem(m->devsysstat), a, 0); n++)
            for(i=0; i<nelem(m->devsysstat); i++)
                m->devsysstat[i] += a[i];
    }
    if(needether(init) && loadbuf(m, &m->etherfd) && readnums(m, nelem(m->netetherstats), a, 1)){
        memmove(m->prevetherstats, m->netetherstats, sizeof m->netetherstats);
        memmove(m->netetherstats, a, sizeof m->netetherstats);
    }
    if(needsignal(init) && loadbuf(m, &m->ifstatsfd) && strncmp(m->buf, "Signal: ", 8)==0 && readnums(m, nelem(
        memmove(m->netetherifstats, a, sizeof m->netetherifstats);
    }
    if(needbattery(init) && loadbuf(m, &m->batteryfd) && readnums(m, nelem(m->batterystats), a, 0))
        memmove(m->batterystats, a, sizeof(m->batterystats));
    if(needbattery(init) && loadbuf(m, &m->bitsybatfd) && readnums(m, 1, a, 0))
        memmove(m->batterystats, a, sizeof(m->batterystats));
    if(needtemp(init) && loadbuf(m, &m->tempfd))
        for(n=0; n < nelem(m->temp) && readnums(m, 2, a, 0); n++)
             m->temp[n] = a[0];
    if(m->remote){
        alarm(0);
        notify(nil);
    }
}
```

⟨*function* memval 56a⟩≡ (67)

```
void
memval(Machine *m, uvlong *v, uvlong *vmax, int)
{
    *v = m->devswap[Mem];
    *vmax = m->devswap[Maxmem];
}
```

⟨*function* swapval 56b⟩≡ (67)

```
void
swapval(Machine *m, uvlong *v, uvlong *vmax, int)
{
    *v = m->devswap[Swap];
    *vmax = m->devswap[Maxswap];
}
```

⟨*function* contextval 56c⟩≡ (67)

```
void
contextval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->devsysstat[Context]-m->prevsysstat[Context];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}
```

⟨*function* intrval 56d⟩≡ (67)

```
/*
 * bug: need to factor in HZ
 */
void
intrval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->devsysstat[Interrupt]-m->prevsysstat[Interrupt];
    *vmax = sleeptime*m->nproc*10;
```

```
    if(init)
        *vmax = sleeptime*10;
}
```

⟨*function* syscallval 57a⟩≡ (67)

```
void
syscallval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->devsysstat[Syscall]-m->prevsysstat[Syscall];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}
```

⟨*function* faultval 57b⟩≡ (67)

```
void
faultval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->devsysstat[Fault]-m->prevsysstat[Fault];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}
```

⟨*function* tlbmissval 57c⟩≡ (67)

```
void
tlbmissval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->devsysstat[TLBfault]-m->prevsysstat[TLBfault];
    *vmax = (sleeptime/1000)*10*m->nproc;
    if(init)
        *vmax = (sleeptime/1000)*10;
}
```

⟨*function* tlbpurgeval 57d⟩≡ (67)

```
void
tlbpurgeval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->devsysstat[TLBpurge]-m->prevsysstat[TLBpurge];
    *vmax = (sleeptime/1000)*10*m->nproc;
    if(init)
        *vmax = (sleeptime/1000)*10;
}
```

⟨*function* loadval 57e⟩≡ (67)

```
void
loadval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->devsysstat[Load];
    *vmax = 1000*m->nproc;
    if(init)
        *vmax = 1000;
}
```

⟨*function* idleval 57f⟩≡ (67)

```
void
idleval(Machine *m, uvlong *v, uvlong *vmax, int)
{
    *v = m->devsysstat[Idle]/m->nproc;
    *vmax = 100;
}
```

⟨*function* inintrval 58a⟩≡ (67)

```
void
inintrval(Machine *m, uvlong *v, uvlong *vmax, int)
{
    *v = m->devsysstat[InIntr]/m->nproc;
    *vmax = 100;
}
```

⟨*function* etherval 58b⟩≡ (67)

```
void
etherval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->netetherstats[In]-m->prevetherstats[In] + m->netetherstats[Out]-m->prevetherstats[Out];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}
```

⟨*function* etherinval 58c⟩≡ (67)

```
void
etherinval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->netetherstats[In]-m->prevetherstats[In];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}
```

⟨*function* etheroutval 58d⟩≡ (67)

```
void
etheroutval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    *v = m->netetherstats[Out]-m->prevetherstats[Out];
    *vmax = sleeptime*m->nproc;
    if(init)
        *vmax = sleeptime;
}
```

⟨*function* ethererrval 58e⟩≡ (67)

```
void
ethererrval(Machine *m, uvlong *v, uvlong *vmax, int init)
{
    int i;

    *v = 0;
    for(i=Err0; i<nelem(m->netetherstats); i++)
        *v += m->netetherstats[i];
    *vmax = (sleeptime/1000)*10*m->nproc;
    if(init)
        *vmax = (sleeptime/1000)*10;
}
```

⟨*function* batteryval 58f⟩≡ (67)

```
void
batteryval(Machine *m, uvlong *v, uvlong *vmax, int)
{
    *v = m->batterystats[0];
    if(m->bitsybatfd >= 0)
        *vmax = 184;  // at least on my bitsy...
    else
        *vmax = 100;
}
```

⟨*function* signalval 59a⟩≡ (67)

```
void
signalval(Machine *m, uvlong *v, uvlong *vmax, int)
{
    ulong l;

    *vmax = sleeptime;
    l = m->netetherifstats[0];
    /*
     * Range is seen to be from about -45 (strong) to -95 (weak); rescale
     */
    if(l == 0){ /* probably not present */
        *v = 0;
        return;
    }
    *v = 20*(l+95);
}
```

⟨*function* tempval 59b⟩≡ (67)

```
void
tempval(Machine *m, uvlong *v, uvlong *vmax, int)
{
    ulong l;

    *vmax = sleeptime;
    l = m->temp[0];
    if(l == ~0 || l == 0)
        *v = 0;
    else
        *v = (l-20)*27;
}
```

⟨*function usage (misc/stats.c)* 59c⟩≡ (67)

```
void
usage(void)
{
    fprint(2, "usage: stats [-O] [-S scale] [-LY] [-%s] [machine...]\n", argchars);
    exits("usage");
}
```

⟨*function* addgraph 59d⟩≡ (67)

```
void
addgraph(int n)
{
    Graph *g, *ograph;
    int i, j;
    static int nadd;

    if(n > nelem(menu2str))
        abort();
    /* avoid two adjacent graphs of same color */
    if(ngraph>0 && graph[ngraph-1].colindex==nadd%Ncolor)
        nadd++;
    ograph = graph;
    graph = emalloc(nmach*(ngraph+1)*sizeof(Graph));
    for(i=0; i<nmach; i++)
        for(j=0; j<ngraph; j++)
            graph[i*(ngraph+1)+j] = ograph[i*ngraph+j];
    free(ograph);
    ngraph++;
```

```
    for(i=0; i<nmach; i++){
        g = &graph[i*ngraph+(ngraph-1)];
        memset(g, 0, sizeof(Graph));
        g->label = menu2str[n]+Opwid;
        g->newvalue = newvaluefn[n];
        g->update = update1; /* no other update functions yet */
        g->mach = &mach[i];
        g->colindex = nadd%Ncolor;
    }
    present[n] = 1;
    nadd++;
}
```

⟨*function* dropgraph 60a⟩≡ (67)

```
void
dropgraph(int which)
{
    Graph *ograph;
    int i, j, n;

    if(which > nelem(menu2str))
        abort();
    /* convert n to index in graph table */
    n = -1;
    for(i=0; i<ngraph; i++)
        if(strcmp(menu2str[which]+Opwid, graph[i].label) == 0){
            n = i;
            break;
        }
    if(n < 0){
        fprint(2, "stats: internal error can't drop graph\n");
        killall("error");
    }
    ograph = graph;
    graph = emalloc(nmach*(ngraph-1)*sizeof(Graph));
    for(i=0; i<nmach; i++){
        for(j=0; j<n; j++)
            graph[i*(ngraph-1)+j] = ograph[i*ngraph+j];
        free(ograph[i*ngraph+j].data);
        freeimage(ograph[i*ngraph+j].overtmp);
        for(j++; j<ngraph; j++)
            graph[i*(ngraph-1)+j-1] = ograph[i*ngraph+j];
    }
    free(ograph);
    ngraph--;
    present[which] = 0;
}
```

⟨*function* addmachine 60b⟩≡ (67)

```
int
addmachine(char *name)
{
    if(ngraph > 0){
        fprint(2, "stats: internal error: ngraph>0 in addmachine()\n");
        usage();
    }
    if(mach == nil)
        nmach = 0; /* a little dance to get us started with local machine by default */
    mach = erealloc(mach, (nmach+1)*sizeof(Machine));
    memset(mach+nmach, 0, sizeof(Machine));
```

```
    if (initmach(mach+nmach, name)){
        nmach++;
        return 1;
    } else
        return 0;
}
```

⟨*function* labelstrs 61a⟩≡ (67)

```
void
labelstrs(Graph *g, char strs[Nlab][Lablen], int *np)
{
    int j;
    uvlong v, vmax;

    g->newvalue(g->mach, &v, &vmax, 1);
    if(logscale){
        for(j=1; j<=2; j++)
            sprint(strs[j-1], "%g", scale*pow(10., j)*(double)vmax/100.);
        *np = 2;
    }else{
        for(j=1; j<=3; j++)
            sprint(strs[j-1], "%g", scale*(double)j*(double)vmax/4.0);
        *np = 3;
    }
}
```

⟨*function* labelwidth 61b⟩≡ (67)

```
int
labelwidth(void)
{
    int i, j, n, w, maxw;
    char strs[Nlab][Lablen];

    maxw = 0;
    for(i=0; i<ngraph; i++){
        /* choose value for rightmost graph */
        labelstrs(&graph[ngraph*(nmach-1)+i], strs, &n);
        for(j=0; j<n; j++){
            w = stringwidth(mediumfont, strs[j]);
            if(w > maxw)
                maxw = w;
        }
    }
    return maxw;
}
```

⟨*function* resize 61c⟩≡ (67)

```
void
resize(void)
{
    int i, j, k, n, startx, starty, x, y, dx, dy, ly, ondata, maxx, wid, nlab;
    Graph *g;
    Rectangle machr, r;
    uvlong v, vmax;
    char buf[128], labs[Nlab][Lablen];

    draw(view, view->r, display->white, nil, ZP);

    /* label left edge */
    x = view->r.min.x;
```

```
y = view->r.min.y + Labspace+mediumfont->height+Labspace;
dy = (view->r.max.y - y)/ngraph;
dx = Labspace+stringwidth(mediumfont, "0")+Labspace;
startx = x+dx+1;
starty = y;
for(i=0; i<ngraph; i++,y+=dy){
    draw(view, Rect(x, y-1, view->r.max.x, y), display->black, nil, ZP);
    draw(view, Rect(x, y, x+dx, view->r.max.y), cols[graph[i].colindex][0], nil, paritypt(x));
    label(Pt(x, y), dy, graph[i].label);
    draw(view, Rect(x+dx, y, x+dx+1, view->r.max.y), cols[graph[i].colindex][2], nil, ZP);
}

/* label top edge */
dx = (view->r.max.x - startx)/nmach;
for(x=startx, i=0; i<nmach; i++,x+=dx){
    draw(view, Rect(x-1, starty-1, x, view->r.max.y), display->black, nil, ZP);
    j = dx/stringwidth(mediumfont, "0");
    n = mach[i].nproc;
    if(n>1 && j>=1+3+mach[i].lgproc){ /* first char of name + (n) */
        j -= 3+mach[i].lgproc;
        if(j <= 0)
            j = 1;
        snprint(buf, sizeof buf, "%.*s(%d)", j, mach[i].shortname, n);
    }else
        snprint(buf, sizeof buf, "%.*s", j, mach[i].shortname);
    string(view, Pt(x+Labspace, view->r.min.y + Labspace), display->black, ZP, mediumfont, buf);
}

maxx = view->r.max.x;

/* label right, if requested */
if(ylabels && dy>Nlab*(mediumfont->height+1)){
    wid = labelwidth();
    if(wid < (maxx-startx)-30){
        /* else there's not enough room */
        maxx -= 1+Lx+wid;
        draw(view, Rect(maxx, starty, maxx+1, view->r.max.y), display->black, nil, ZP);
        y = starty;
        for(j=0; j<ngraph; j++, y+=dy){
            /* choose value for rightmost graph */
            g = &graph[ngraph*(nmach-1)+j];
            labelstrs(g, labs, &nlab);
            r = Rect(maxx+1, y, view->r.max.x, y+dy-1);
            if(j == ngraph-1)
                r.max.y = view->r.max.y;
            draw(view, r, cols[g->colindex][0], nil, paritypt(r.min.x));
            for(k=0; k<nlab; k++){
                ly = y + (dy*(nlab-k)/(nlab+1));
                draw(view, Rect(maxx+1, ly, maxx+1+Lx, ly+1), display->black, nil, ZP);
                ly -= mediumfont->height/2;
                string(view, Pt(maxx+1+Lx, ly), display->black, ZP, mediumfont, labs[k]);
            }
        }
    }
}

/* create graphs */
for(i=0; i<nmach; i++){
    machr = Rect(startx+i*dx, starty, maxx, view->r.max.y);
    if(i < nmach-1)
```

```
            machr.max.x = startx+(i+1)*dx - 1;
        y = starty;
        for(j=0; j<ngraph; j++, y+=dy){
            g = &graph[i*ngraph+j];
            /* allocate data */
            ondata = g->ndata;
            g->ndata = Dx(machr)+1; /* may be too many if label will be drawn here; so what? */
            g->data = erealloc(g->data, g->ndata*sizeof(g->data[0]));
            if(g->ndata > ondata)
                memset(g->data+ondata, 0, (g->ndata-ondata)*sizeof(g->data[0]));
            /* set geometry */
            g->r = machr;
            g->r.min.y = y;
            g->r.max.y = y+dy - 1;
            if(j == ngraph-1)
                g->r.max.y = view->r.max.y;
            draw(view, g->r, cols[g->colindex][0], nil, paritypt(g->r.min.x));
            g->overflow = 0;
            r = g->r;
            r.max.y = r.min.y+mediumfont->height;
            r.max.x = r.min.x+stringwidth(mediumfont, "999999999999");
            freeimage(g->overtmp);
            g->overtmp = nil;
            if(r.max.x <= g->r.max.x)
                g->overtmp = allocimage(display, r, view->chan, 0, -1);
            g->newvalue(g->mach, &v, &vmax, 0);
            redraw(g, vmax);
        }
    }

    flushimage(display, 1);
}
```

⟨*function* eresized 63a⟩≡ (67)

```
void
eresized(int new)
{
    lockdisplay(display);
    if(new && getwindow(display, Refnone) < 0) {
        fprint(2, "stats: can't reattach to window\n");
        killall("reattach");
    }
    resize();
    unlockdisplay(display);
}
```

⟨*function* mouseproc 63b⟩≡ (67)

```
void
mouseproc(void)
{
    Mouse mouse;
    int i;

    for(;;){
        mouse = emouse();
        if(mouse.buttons == 4){
            lockdisplay(display);
            for(i=0; i<Nmenu2; i++)
                if(present[i])
                    memmove(menu2str[i], "drop ", Opwid);
```

```
                else
                    memmove(menu2str[i], "add  ", Opwid);
            i = emenuhit(3, &mouse, &menu2);
            if(i >= 0){
                if(!present[i])
                    addgraph(i);
                else if(ngraph > 1)
                    dropgraph(i);
                resize();
            }
            unlockdisplay(display);
        }
    }
}
```

⟨*function* startproc 64a⟩≡ (67)

```
void
startproc(void (*f)(void), int index)
{
    int pid;

    switch(pid = rfork(RFPROC|RFMEM|RFNOWAIT)){
    case -1:
        fprint(2, "stats: fork failed: %r\n");
        killall("fork failed");
    case 0:
        f();
        fprint(2, "stats: %s process exits\n", procnames[index]);
        if(index >= 0)
            killall("process died");
        exits(nil);
    }
    if(index >= 0)
        pids[index] = pid;
}
```

⟨*function main (misc/stats.c)* 64b⟩≡ (67)

```
void
main(int argc, char *argv[])
{
    int i, j;
    double secs;
    uvlong v, vmax, nargs;
    char args[100];

    nmach = 1;
    mysysname = getenv("sysname");
    if(mysysname == nil){
        fprint(2, "stats: can't find $sysname: %r\n");
        exits("sysname");
    }
    mysysname = estrdup(mysysname);

    nargs = 0;
    ARGBEGIN{
    case 'T':
        secs = atof(EARGF(usage()));
        if(secs > 0)
            sleeptime = 1000*secs;
        break;
```

```
case 'S':
    scale = atof(EARGF(usage()));
    if(scale <= 0)
        usage();
    break;
case 'L':
    logscale++;
    break;
case 'Y':
    ylabels++;
    break;
case 'O':
    oldsystem = 1;
    break;
default:
    if(nargs>=sizeof args || strchr(argchars, ARGC())==nil)
        usage();
    args[nargs++] = ARGC();
}ARGEND

if(argc == 0){
    mach = emalloc(nmach*sizeof(Machine));
    initmach(&mach[0], mysysname);
    readmach(&mach[0], 1);
}else{
    for(i=j=0; i<argc; i++){
        if (addmachine(argv[i]))
            readmach(&mach[j++], 1);
    }
    if (j == 0)
        exits("connect");
}

for(i=0; i<nargs; i++)
switch(args[i]){
default:
    fprint(2, "stats: internal error: unknown arg %c\n", args[i]);
    usage();
case 'b':
    addgraph(Mbattery);
    break;
case 'c':
    addgraph(Mcontext);
    break;
case 'e':
    addgraph(Mether);
    break;
case 'E':
    addgraph(Metherin);
    addgraph(Metherout);
    break;
case 'f':
    addgraph(Mfault);
    break;
case 'i':
    addgraph(Mintr);
    break;
case 'I':
    addgraph(Mload);
    addgraph(Midle);
```

```
        addgraph(Minintr);
        break;
    case 'l':
        addgraph(Mload);
        break;
    case 'm':
        addgraph(Mmem);
        break;
    case 'n':
        addgraph(Metherin);
        addgraph(Metherout);
        addgraph(Methererr);
        break;
    case 'p':
        addgraph(Mtlbpurge);
        break;
    case 's':
        addgraph(Msyscall);
        break;
    case 't':
        addgraph(Mtlbmiss);
        addgraph(Mtlbpurge);
        break;
    case '8':
        addgraph(Msignal);
        break;
    case 'w':
        addgraph(Mswap);
        break;
    case 'z':
        addgraph(Mtemp);
        break;
    }

    if(ngraph == 0)
        addgraph(Mload);

    for(i=0; i<nmach; i++)
        for(j=0; j<ngraph; j++)
            graph[i*ngraph+j].mach = &mach[i];

    if(initdraw(nil, nil, "stats") < 0){
        fprint(2, "stats: initdraw failed: %r\n");
        exits("initdraw");
    }
    colinit();
    einit(Emouse);
    notify(nil);
    startproc(mouseproc, Mouseproc);
    pids[Mainproc] = getpid();
    display->locking = 1; /* tell library we're using the display lock */

    resize();

    unlockdisplay(display); /* display is still locked from initdraw() */
    for(;;){
        for(i=0; i<nmach; i++)
            readmach(&mach[i], 0);
        lockdisplay(display);
        parity = 1-parity;
```

```
        for(i=0; i<nmach*ngraph; i++){
            graph[i].newvalue(graph[i].mach, &v, &vmax, 0);
            graph[i].update(&graph[i], v, vmax);
        }
        flushimage(display, 1);
        unlockdisplay(display);
        sleep(sleeptime);
    }
}
```

⟨misc/stats.c 67⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>
#include <auth.h>
#include <fcall.h>
#include <draw.h>
#include <window.h>
#include <event.h>

⟨constant MAXNUM 42a⟩

typedef struct Graph Graph;
typedef struct Machine Machine;

⟨struct Graph 42b⟩

⟨enum _anon_ (misc/stats.c) 42c⟩

⟨struct Machine 42d⟩

⟨enum _anon_ (misc/stats.c)2 43a⟩

⟨enum _anon_ (misc/stats.c)3 43b⟩

⟨enum Menu2 43c⟩

⟨global menu2str 44a⟩


void contextval(Machine*, uvlong*, uvlong*, int),
    etherval(Machine*, uvlong*, uvlong*, int),
    ethererrval(Machine*, uvlong*, uvlong*, int),
    etherinval(Machine*, uvlong*, uvlong*, int),
    etheroutval(Machine*, uvlong*, uvlong*, int),
    faultval(Machine*, uvlong*, uvlong*, int),
    intrval(Machine*, uvlong*, uvlong*, int),
    inintrval(Machine*, uvlong*, uvlong*, int),
    loadval(Machine*, uvlong*, uvlong*, int),
    idleval(Machine*, uvlong*, uvlong*, int),
    memval(Machine*, uvlong*, uvlong*, int),
    swapval(Machine*, uvlong*, uvlong*, int),
    syscallval(Machine*, uvlong*, uvlong*, int),
    tlbmissval(Machine*, uvlong*, uvlong*, int),
    tlbpurgeval(Machine*, uvlong*, uvlong*, int),
    batteryval(Machine*, uvlong*, uvlong*, int),
    signalval(Machine*, uvlong*, uvlong*, int),
    tempval(Machine*, uvlong*, uvlong*, int);

⟨global menu2 44b⟩
```

⟨*global* `present` 44c⟩
⟨*global* `newvaluefn` 44d⟩

⟨*global cols (misc/stats.c)* 44e⟩
⟨*global* `graph` 45a⟩
⟨*global* `mach` 45b⟩
⟨*global* `mediumfont` 45c⟩
⟨*global* `mysysname` 45d⟩
⟨*global* `argchars` 45e⟩
⟨*global* `pids` 45f⟩
⟨*global* `parity` 45g⟩
⟨*global* `nmach` 45h⟩
⟨*global* `ngraph` 45i⟩
⟨*global* `scale` 45j⟩
⟨*global* `logscale` 45k⟩
⟨*global* `ylabels` 45l⟩
⟨*global* `oldsystem` 45m⟩
⟨*global* `sleeptime` 45n⟩

⟨*global* `procnames` 45o⟩

⟨*function* `killall` 45p⟩

⟨*function* `emalloc` 46a⟩

⟨*function* `erealloc` 46b⟩

⟨*function* `estrdup` 46c⟩

⟨*function mkcol (misc/stats.c)* 46d⟩

⟨*function colinit (misc/stats.c)* 46e⟩

⟨*function* `loadbuf` 47a⟩

⟨*function* `label` 47b⟩

⟨*function* `paritypt` 47c⟩

⟨*function* `datapoint` 48a⟩

⟨*function* `drawdatum` 48b⟩

⟨*function redraw (misc/stats.c)* 49a⟩

⟨*function* `update1` 49b⟩

⟨*function* `readnums` 49c⟩

⟨*function* `filter` 50a⟩

⟨*function* `connect9fs` 50b⟩

⟨*function* `old9p` 51a⟩

⟨*function* `connectexportfs` 51b⟩

⟨*function* `readswap` 52a⟩

⟨*function* `shortname` 52b⟩

⟨*function* `ilog10` 52c⟩

⟨*function* `initmach` 53⟩

⟨*global* `catchalarm` 54a⟩

⟨*function* `alarmed` 54b⟩

⟨*function* `needswap` 54c⟩

⟨*function* `needstat` 54d⟩

⟨*function* `needether` 55a⟩

⟨*function* `needbattery` 55b⟩

⟨*function* `needsignal` 55c⟩

⟨*function* `needtemp` 55d⟩

⟨*function* `readmach` 55e⟩

⟨*function* `memval` 56a⟩

⟨*function* `swapval` 56b⟩

⟨*function* `contextval` 56c⟩

⟨*function* `intrval` 56d⟩

⟨*function* `syscallval` 57a⟩

⟨*function* `faultval` 57b⟩

⟨*function* `tlbmissval` 57c⟩

⟨*function* `tlbpurgeval` 57d⟩

⟨*function* `loadval` 57e⟩

⟨*function* `idleval` 57f⟩

⟨*function* `inintrval` 58a⟩

⟨*function* `etherval` 58b⟩

⟨*function* `etherinval` 58c⟩

⟨*function* `etheroutval` 58d⟩

⟨*function* `ethererrval` 58e⟩

⟨*function* `batteryval` 58f⟩

⟨*function* `signalval` 59a⟩

⟨*function* tempval 59b⟩

⟨*function usage (misc/stats.c)* 59c⟩

⟨*function* addgraph 59d⟩

⟨*function* dropgraph 60a⟩

⟨*function* addmachine 60b⟩

⟨*function* labelstrs 61a⟩

⟨*function* labelwidth 61b⟩

⟨*function* resize 61c⟩

⟨*function* eresized 63a⟩

⟨*function* mouseproc 63b⟩

⟨*function* startproc 64a⟩

⟨*function main (misc/stats.c)* 64b⟩

D.2 iostats/

D.2.1 iostats/statfs.h

⟨*constant* DEBUGFILE 70a⟩≡ (72d)

```
/*
 * statfs.h - definitions for statistic gathering file server
 */

#define DEBUGFILE "iostats.out"
```

⟨*constant* DONESTR 70b⟩≡ (72d)

```
#define DONESTR  "done"
```

⟨*constant* DEBUG 70c⟩≡ (72d)

```
#define DEBUG  if(dbg)fprint
```

⟨*constant* MAXPROC 70d⟩≡ (72d)

```
#define MAXPROC  16
```

⟨*constant* FHASHSIZE 70e⟩≡ (72d)

```
#define FHASHSIZE 64
```

⟨*function* fidhash 70f⟩≡ (72d)

```
#define fidhash(s) fhash[s%FHASHSIZE]
```

⟨*enum* _anon_ (iostats/statfs.h) 70g⟩≡ (72d)

```
enum{
    Maxfdata = 8192, /* max size of data in 9P message */
    Maxrpc  = 20000,/* number of RPCs we'll log */
};
```

⟨*struct* Frec 71a⟩≡ (72d)

```
struct Frec
{
    Frec *next;
    char *op;
    ulong nread;
    ulong nwrite;
    ulong bread;
    ulong bwrite;
    ulong opens;
};
```

⟨*struct* Rpc 71b⟩≡ (72d)

```
struct Rpc
{
    char *name;
    ulong count;
    vlong time;
    vlong lo;
    vlong hi;
    ulong bin;
    ulong bout;
};
```

⟨*struct* Stats 71c⟩≡ (72d)

```
struct Stats
{
    ulong totread;
    ulong totwrite;
    ulong nrpc;
    ulong nproto;
    Rpc rpc[Maxrpc];
};
```

⟨*struct* Fsrpc 71d⟩≡ (72d)

```
struct Fsrpc
{
    int busy;   /* Work buffer has pending rpc to service */
    uintptr pid;   /* Pid of slave process executing the rpc */
    int canint;   /* Interrupt gate */
    int flushtag;  /* Tag on which to reply to flush */
    Fcall work;   /* Plan 9 incoming Fcall */
    uchar buf[IOHDRSZ+Maxfdata]; /* Data buffer */
};
```

⟨*struct* Fid 71e⟩≡ (72d)

```
struct Fid
{
    int fid;   /* system fd for i/o */
    File *f;   /* File attached to this fid */
    int mode;
    int nr;   /* fid number */
    Fid *next;   /* hash link */
    ulong nread;
    ulong nwrite;
    ulong bread;
    ulong bwrite;
    vlong offset;   /* for directories */
};
```

⟨*struct* File 72a⟩≡ (72d)

```
struct File
{
    char *name;
    Qid qid;
    int inval;
    File *parent;
    File *child;
    File *childlist;
};
```

⟨*struct* Proc 72b⟩≡ (72d)

```
struct Proc
{
    uintptr pid;
    int busy;
    Proc *next;
};
```

⟨*enum* _anon_ (iostats/statfs.h)2 72c⟩≡ (72d)

```
enum
{
    Nr_workbufs  = 40,
    Dsegpad  = 8192,
    Fidchunk = 1000,
};
```

⟨iostats/statfs.h 72d⟩≡

```
⟨constant DEBUGFILE 70a⟩
⟨constant DONESTR 70b⟩
⟨constant DEBUG 70c⟩
⟨constant MAXPROC 70d⟩
⟨constant FHASHSIZE 70e⟩
⟨function fidhash 70f⟩

⟨enum _anon_ (iostats/statfs.h) 70g⟩

typedef struct Fsrpc Fsrpc;
typedef struct Fid Fid;
typedef struct File File;
typedef struct Proc Proc;
typedef struct Stats Stats;
typedef struct Rpc Rpc;
typedef struct Frec Frec;

⟨struct Frec 71a⟩

⟨struct Rpc 71b⟩

⟨struct Stats 71c⟩

⟨struct Fsrpc 71d⟩

⟨struct Fid 71e⟩

⟨struct File 72a⟩

⟨struct Proc 72b⟩

⟨enum _anon_ (iostats/statfs.h)2 72c⟩
```

```
extern Fsrpc *Workq;
extern int   dbg;
extern File *root;
extern Fid **fhash;
extern Fid *fidfree;
extern int qid;
extern Proc *Proclist;
extern int done;
extern Stats *stats;
extern Frec *frhead;
extern Frec *frtail;
extern int myiounit;

/* File system protocol service procedures */
void Xcreate(Fsrpc*), Xclunk(Fsrpc*);
void Xversion(Fsrpc*), Xauth(Fsrpc*), Xflush(Fsrpc*);
void Xattach(Fsrpc*), Xwalk(Fsrpc*), Xauth(Fsrpc*);
void Xremove(Fsrpc*), Xstat(Fsrpc*), Xwstat(Fsrpc*);
void slave(Fsrpc*);

void reply(Fcall*, Fcall*, char*);
Fid  *getfid(int);
int freefid(int);
Fid *newfid(int);
Fsrpc *getsbuf(void);
void initroot(void);
void fatal(char*);
void makepath(char*, File*, char*);
File *file(File*, char*);
void slaveopen(Fsrpc*);
void slaveread(Fsrpc*);
void slavewrite(Fsrpc*);
void blockingslave(void);
void reopen(Fid *f);
void noteproc(int, char*);
void flushaction(void*, char*);
void catcher(void*, char*);
ulong msec(void);
void fidreport(Fid*);
```

D.2.2 iostats/globals.c

⟨*global* Workq 73a⟩≡ (74g)

```
Fsrpc *Workq;
```

⟨*global* dbg 73b⟩≡ (74g)

```
int   dbg;
```

⟨*global* root 73c⟩≡ (74g)

```
File *root;
```

⟨*global* fhash 73d⟩≡ (74g)

```
Fid **fhash;
```

⟨*global* fidfree 73e⟩≡ (74g)

```
Fid *fidfree;
```

⟨*global* qid 73f⟩≡ (74g)

```
int qid;
```

⟨*global* Proclist 74a⟩≡ (74g)
```
Proc *Proclist;
```

⟨*global* done 74b⟩≡ (74g)
```
int done;
```

⟨*global* stats 74c⟩≡ (74g)
```
Stats *stats;
```

⟨*global* frhead 74d⟩≡ (74g)
```
Frec *frhead;
```

⟨*global* frtail 74e⟩≡ (74g)
```
Frec *frtail;
```

⟨*global* myiounit 74f⟩≡ (74g)
```
int myiounit;
```

⟨iostats/globals.c 74g⟩≡
```
#include <u.h>
#include <libc.h>
#include <auth.h>
#include <fcall.h>

#include "statfs.h"

⟨global Workq 73a⟩
⟨global dbg 73b⟩
⟨global root 73c⟩
⟨global fhash 73d⟩
⟨global fidfree 73e⟩
⟨global qid 73f⟩
⟨global Proclist 74a⟩
⟨global done 74b⟩
⟨global stats 74c⟩
⟨global frhead 74d⟩
⟨global frtail 74e⟩
⟨global myiounit 74f⟩
```

D.2.3 iostats/iostats.c

⟨*global* fcalls 74h⟩≡ (84b)
```
void (*fcalls[])(Fsrpc*) =
{
    [Tversion] Xversion,
    [Tauth] Xauth,
    [Tflush] Xflush,
    [Tattach] Xattach,
    [Twalk]  Xwalk,
    [Topen]  slave,
    [Tcreate] Xcreate,
    [Tclunk] Xclunk,
    [Tread]  slave,
    [Twrite] slave,
    [Tremove] Xremove,
    [Tstat]  Xstat,
    [Twstat] Xwstat,
};
```

⟨*global* p 75a⟩≡ (84b)

```
int p[2];
```

⟨*function usage (iostats/iostats.c)* 75b⟩≡ (84b)

```
static void
usage(void)
{
    fprint(2, "usage: iostats [-d] [-f debugfile] cmds [args ...]\n");
    exits("usage");
}
```

⟨*function main (iostats/iostats.c)* 75c⟩≡ (84b)

```
void
main(int argc, char **argv)
{
    Fsrpc *r;
    Rpc *rpc;
    Proc *m;
    Frec *fr;
    Fid *fid;
    ulong ttime;
    char *dbfile, *s;
    char buf[128];
    float brpsec, bwpsec, bppsec;
    int type, cpid, fspid, n;

    dbfile = DEBUGFILE;

    ARGBEGIN{
    case 'd':
        dbg++;
        break;
    case 'f':
        dbfile = ARGF();
        break;
    default:
        usage();
    }ARGEND

    if(argc == 0)
        usage();

    if(dbg) {
        close(2);
        create(dbfile, OWRITE, 0666);
    }

    if(pipe(p) < 0)
        fatal("pipe");

    switch(cpid = fork()) {
    case -1:
        fatal("fork");
    case 0:
        close(p[1]);
        if(getwd(buf, sizeof(buf)) == 0)
            fatal("no working directory");

        rfork(RFENVG|RFNAMEG|RFNOTEG);
        if(mount(p[0], -1, "/", MREPL, "") < 0)
```

```
        fatal("mount /");

    bind("#c/pid", "/dev/pid", MREPL);
    bind("#e", "/env", MREPL|MCREATE);
    close(0);
    close(1);
    close(2);
    open("/fd/0", OREAD);
    open("/fd/1", OWRITE);
    open("/fd/2", OWRITE);

    if(chdir(buf) < 0)
        fatal("chdir");

    runprog(argv);
default:
    close(p[0]);
}

switch(fspid = fork()) {
default:
    while(cpid != waitpid())
        ;
    postnote(PNPROC, fspid, DONESTR);
    while(fspid != waitpid())
        ;
    exits(0);
case -1:
    fatal("fork");
case 0:
    break;
}

/* Allocate work queues in shared memory */
malloc(Dsegpad);
Workq = malloc(sizeof(Fsrpc)*Nr_workbufs);
stats = malloc(sizeof(Stats));
fhash = mallocz(sizeof(Fid*)*FHASHSIZE, 1);

if(Workq == 0 || fhash == 0 || stats == 0)
    fatal("no initial memory");

memset(Workq, 0, sizeof(Fsrpc)*Nr_workbufs);
memset(stats, 0, sizeof(Stats));

stats->rpc[Tversion].name = "version";
stats->rpc[Tauth].name = "auth";
stats->rpc[Tflush].name = "flush";
stats->rpc[Tattach].name = "attach";
stats->rpc[Twalk].name = "walk";
stats->rpc[Topen].name = "open";
stats->rpc[Tcreate].name = "create";
stats->rpc[Tclunk].name = "clunk";
stats->rpc[Tread].name = "read";
stats->rpc[Twrite].name = "write";
stats->rpc[Tremove].name = "remove";
stats->rpc[Tstat].name = "stat";
stats->rpc[Twstat].name = "wstat";

for(n = 0; n < Maxrpc; n++)
```

```
    stats->rpc[n].lo = 10000000000LL;

fmtinstall('M', dirmodefmt);
fmtinstall('D', dirfmt);
fmtinstall('F', fcallfmt);

if(chdir("/") < 0)
    fatal("chdir");

initroot();

DEBUG(2, "statfs: %s\n", buf);

notify(catcher);

for(;;) {
    r = getsbuf();
    if(r == 0)
        fatal("Out of service buffers");

    n = read9pmsg(p[1], r->buf, sizeof(r->buf));
    if(done)
        break;
    if(n < 0)
        fatal("read server");

    if(convM2S(r->buf, n, &r->work) == 0)
        fatal("format error");

    stats->nrpc++;
    stats->nproto += n;

    DEBUG(2, "%F\n", &r->work);

    type = r->work.type;
    rpc = &stats->rpc[type];
    rpc->count++;
    rpc->bin += n;
    (fcalls[type])(r);
}

/* Clear away the slave children */
for(m = Proclist; m; m = m->next)
    postnote(PNPROC, m->pid, "kill");

rpc = &stats->rpc[Tread];
brpsec = (float)stats->totread / (((float)rpc->time/1e9)+.000001);

rpc = &stats->rpc[Twrite];
bwpsec = (float)stats->totwrite / (((float)rpc->time/1e9)+.000001);

ttime = 0;
for(n = 0; n < Maxrpc; n++) {
    rpc = &stats->rpc[n];
    if(rpc->count == 0)
        continue;
    ttime += rpc->time;
}

bppsec = (float)stats->nproto / ((ttime/1e9)+.000001);
```

```
    fprint(2, "\nread      %lud bytes, %g Kb/sec\n", stats->totread, brpsec/1024.0);
    fprint(2, "write     %lud bytes, %g Kb/sec\n", stats->totwrite, bwpsec/1024.0);
    fprint(2, "protocol  %lud bytes, %g Kb/sec\n", stats->nproto, bppsec/1024.0);
    fprint(2, "rpc       %lud count\n\n", stats->nrpc);

    fprint(2, "%-10s %5s %5s %5s %5s %5s          T       R\n",
          "Message", "Count", "Low", "High", "Time", "Averg");

    for(n = 0; n < Maxrpc; n++) {
        rpc = &stats->rpc[n];
        if(rpc->count == 0)
            continue;
        fprint(2, "%-10s %5lud %5llud %5llud %5llud %5llud ms %8lud %8lud bytes\n",
            rpc->name,
            rpc->count,
            rpc->lo/1000000,
            rpc->hi/1000000,
            rpc->time/1000000,
            rpc->time/1000000/rpc->count,
            rpc->bin,
            rpc->bout);
    }

    for(n = 0; n < FHASHSIZE; n++)
        for(fid = fhash[n]; fid; fid = fid->next)
            if(fid->nread || fid->nwrite)
                fidreport(fid);
    if(frhead == 0)
        exits(0);

    fprint(2, "\nOpens    Reads  (bytes)   Writes  (bytes) File\n");
    for(fr = frhead; fr; fr = fr->next) {
        s = fr->op;
        if(*s) {
            if(strcmp(s, "/fd/0") == 0)
                s = "(stdin)";
            else
            if(strcmp(s, "/fd/1") == 0)
                s = "(stdout)";
            else
            if(strcmp(s, "/fd/2") == 0)
                s = "(stderr)";
        }
        else
            s = "/.";

        fprint(2, "%5lud %8lud %8lud %8lud %8lud %s\n", fr->opens, fr->nread, fr->bread,
                            fr->nwrite, fr->bwrite, s);
    }

    exits(0);
}
```

⟨*function* reply 78⟩≡ (84b)

```
void
reply(Fcall *r, Fcall *t, char *err)
{
    uchar data[IOHDRSZ+Maxfdata];
    int n;
```

```
    t->tag = r->tag;
    t->fid = r->fid;
    if(err) {
        t->type = Rerror;
        t->ename = err;
    }
    else
        t->type = r->type + 1;

    DEBUG(2, "\t%F\n", t);

    n = convS2M(t, data, sizeof data);
    if(write(p[1], data, n)!=n)
        fatal("mount write");
    stats->nproto += n;
    stats->rpc[t->type-1].bout += n;
}
```

⟨*function* getfid 79a⟩≡ (84b)

```
Fid *
getfid(int nr)
{
    Fid *f;

    for(f = fidhash(nr); f; f = f->next)
        if(f->nr == nr)
            return f;

    return 0;
}
```

⟨*function* freefid 79b⟩≡ (84b)

```
int
freefid(int nr)
{
    Fid *f, **l;

    l = &fidhash(nr);
    for(f = *l; f; f = f->next) {
        if(f->nr == nr) {
            *l = f->next;
            f->next = fidfree;
            fidfree = f;
            return 1;
        }
        l = &f->next;
    }

    return 0;
}
```

⟨*function* newfid 79c⟩≡ (84b)

```
Fid *
newfid(int nr)
{
    Fid *new, **l;
    int i;

    l = &fidhash(nr);
```

```
    for(new = *l; new; new = new->next)
        if(new->nr == nr)
            return 0;

    if(fidfree == 0) {
        fidfree = mallocz(sizeof(Fid) * Fidchunk, 1);
        if(fidfree == 0)
            fatal("out of memory");

        for(i = 0; i < Fidchunk-1; i++)
            fidfree[i].next = &fidfree[i+1];

        fidfree[Fidchunk-1].next = 0;
    }

    new = fidfree;
    fidfree = new->next;

    memset(new, 0, sizeof(Fid));
    new->next = *l;
    *l = new;
    new->nr = nr;
    new->fid = -1;
    new->nread = 0;
    new->nwrite = 0;
    new->bread = 0;
    new->bwrite = 0;

    return new;
}
```

⟨*function* getsbuf 80a⟩≡ (84b)

```
Fsrpc *
getsbuf(void)
{
    static int ap;
    int look;
    Fsrpc *wb;

    for(look = 0; look < Nr_workbufs; look++) {
        if(++ap == Nr_workbufs)
            ap = 0;
        if(Workq[ap].busy == 0)
            break;
    }

    if(look == Nr_workbufs)
        fatal("No more work buffers");

    wb = &Workq[ap];
    wb->pid = 0;
    wb->canint = 0;
    wb->flushtag = NOTAG;
    wb->busy = 1;

    return wb;
}
```

⟨*function* strcatalloc 80b⟩≡ (84b)

```
char *
```

```
strcatalloc(char *p, char *n)
{
    char *v;

    v = realloc(p, strlen(p)+strlen(n)+1);
    if(v == 0)
        fatal("no memory");
    strcat(v, n);
    return v;
}
```

⟨*function* file 81a⟩≡ (84b)

```
File *
file(File *parent, char *name)
{
    char buf[128];
    File *f, *new;
    Dir *dir;

    DEBUG(2, "\tfile: 0x%p %s name %s\n", parent, parent->name, name);

    for(f = parent->child; f; f = f->childlist)
        if(strcmp(name, f->name) == 0)
            break;

    if(f != nil && !f->inval)
        return f;
    makepath(buf, parent, name);
    dir = dirstat(buf);
    if(dir == nil)
        return 0;
    if(f != nil){
        free(dir);
        f->inval = 0;
        return f;
    }

    new = malloc(sizeof(File));
    if(new == 0)
        fatal("no memory");

    memset(new, 0, sizeof(File));
    new->name = strdup(name);
    if(new->name == nil)
        fatal("can't strdup");
    new->qid.type = dir->qid.type;
    new->qid.vers = dir->qid.vers;
    new->qid.path = ++qid;

    new->parent = parent;
    new->childlist = parent->child;
    parent->child = new;

    free(dir);
    return new;
}
```

⟨*function* initroot 81b⟩≡ (84b)

```
void
initroot(void)
```

```
{
    Dir *dir;

    root = malloc(sizeof(File));
    if(root == 0)
        fatal("no memory");

    memset(root, 0, sizeof(File));
    root->name = strdup("/");
    if(root->name == nil)
        fatal("can't strdup");
    dir = dirstat(root->name);
    if(dir == nil)
        fatal("root stat");

    root->qid.type = dir->qid.type;
    root->qid.vers = dir->qid.vers;
    root->qid.path = ++qid;
    free(dir);
}
```

⟨*function* makepath 82a⟩≡ (84b)

```
void
makepath(char *as, File *p, char *name)
{
    char *c, *seg[100];
    int i;
    char *s;

    seg[0] = name;
    for(i = 1; i < 100 && p; i++, p = p->parent){
        seg[i] = p->name;
        if(strcmp(p->name, "/") == 0)
            seg[i] = ""; /* will insert slash later */
    }

    s = as;
    while(i--) {
        for(c = seg[i]; *c; c++)
            *s++ = *c;
        *s++ = '/';
    }
    while(s[-1] == '/')
        s--;
    *s = '\0';
    if(as == s) /* empty string is root */
        strcpy(as, "/");
}
```

⟨*function* fatal 82b⟩≡ (84b)

```
void
fatal(char *s)
{
    Proc *m;

    fprint(2, "iostats: %s: %r\n", s);

    /* Clear away the slave children */
    for(m = Proclist; m; m = m->next)
        postnote(PNPROC, m->pid, "exit");
```

```
    exits("fatal");
}
```

⟨*function* rdenv 83a⟩≡ (84b)

```
char*
rdenv(char *v, char **end)
{
    int fd, n;
    char *buf;
    Dir *d;
    if((fd = open(v, OREAD)) == -1)
        return nil;
    d = dirfstat(fd);
    if(d == nil || (buf = malloc(d->length + 1)) == nil)
        return nil;
    n = (int)d->length;
    n = read(fd, buf, n);
    close(fd);
    if(n <= 0){
        free(buf);
        buf = nil;
    }else{
        if(buf[n-1] != '\0')
            buf[n++] = '\0';
        *end = &buf[n];
    }
    free(d);
    return buf;
}
```

⟨*global* Defaultpath 83b⟩≡ (84b)

```
char Defaultpath[] = ".\0/bin";
```

⟨*function* runprog 83c⟩≡ (84b)

```
void
runprog(char *argv[])
{
    char *path, *ep, *p;
    char arg0[256];

    path = rdenv("/env/path", &ep);
    if(path == nil){
        path = Defaultpath;
        ep = path+sizeof(Defaultpath);
    }
    for(p = path; p < ep; p += strlen(p)+1){
        snprint(arg0, sizeof arg0, "%s/%s", p, argv[0]);
        exec(arg0, argv);
    }
    fatal("exec");
}
```

⟨*function* catcher 83d⟩≡ (84b)

```
void
catcher(void *a, char *msg)
{
    USED(a);
    if(strcmp(msg, DONESTR) == 0) {
        done = 1;
```

```
        noted(NCONT);
    }
    if(strcmp(msg, "exit") == 0)
        exits("exit");

    noted(NDFLT);
}
```

⟨*function* fidreport 84a⟩≡ (84b)

```
void
fidreport(Fid *f)
{
    char *p, path[128];
    Frec *fr;

    p = path;
    makepath(p, f->f, "");

    for(fr = frhead; fr; fr = fr->next) {
        if(strcmp(fr->op, p) == 0) {
            fr->nread += f->nread;
            fr->nwrite += f->nwrite;
            fr->bread += f->bread;
            fr->bwrite += f->bwrite;
            fr->opens++;
            return;
        }
    }

    fr = malloc(sizeof(Frec));
    if(fr == 0 || (fr->op = strdup(p)) == 0)
        fatal("no memory");

    fr->nread = f->nread;
    fr->nwrite = f->nwrite;
    fr->bread = f->bread;
    fr->bwrite = f->bwrite;
    fr->opens = 1;
    if(frhead == 0) {
        frhead = fr;
        frtail = fr;
    }
    else {
        frtail->next = fr;
        frtail = fr;
    }
    fr->next = 0;
}
```

⟨iostats/iostats.c 84b⟩≡

```
/*
 * iostats - Gather file system information
 */
#include <u.h>
#include <libc.h>
#include <auth.h>
#include <fcall.h>

#include "statfs.h"
```

```
void runprog(char**);
```

⟨*global* `fcalls` 74h⟩

⟨*global* `p` 75a⟩

⟨*function usage (iostats/iostats.c)* 75b⟩

⟨*function main (iostats/iostats.c)* 75c⟩

⟨*function* `reply` 78⟩

⟨*function* `getfid` 79a⟩

⟨*function* `freefid` 79b⟩

⟨*function* `newfid` 79c⟩

⟨*function* `getsbuf` 80a⟩

⟨*function* `strcatalloc` 80b⟩

⟨*function* `file` 81a⟩

⟨*function* `initroot` 81b⟩

⟨*function* `makepath` 82a⟩

⟨*function* `fatal` 82b⟩

⟨*function* `rdenv` 83a⟩

⟨*global* `Defaultpath` 83b⟩
⟨*function* `runprog` 83c⟩

⟨*function* `catcher` 83d⟩

⟨*function* `fidreport` 84a⟩

D.2.4 iostats/statsrv.c

⟨*global* `Ebadfid` 85a⟩≡ (97c)
```
char Ebadfid[] = "Bad fid";
```

⟨*global* `Enotdir` 85b⟩≡ (97c)
```
char Enotdir[] ="Not a directory";
```

⟨*global* `Edupfid` 85c⟩≡ (97c)
```
char Edupfid[] = "Fid already in use";
```

⟨*global* `Eopen` 85d⟩≡ (97c)
```
char Eopen[] = "Fid already opened";
```

⟨*global* `Exmnt` 85e⟩≡ (97c)
```
char Exmnt[] = "Cannot .. past mount point";
```

⟨*global* `Enoauth` 85f⟩≡ (97c)
```
char Enoauth[] = "iostats: Authentication failed";
```

⟨*global* Ebadver 86a⟩≡ (97c)

```
char Ebadver[] = "Unrecognized 9P version";
```

⟨*function* okfile 86b⟩≡ (97c)

```
int
okfile(char *s, int mode)
{
    if(strncmp(s, "/fd/", 3) == 0){
        /* 0, 1, and 2 we handle ourselves */
        if(s[4]=='/' || atoi(s+4) > 2)
            return 0;
        return 1;
    }
    if(strncmp(s, "/net/ssl", 8) == 0)
        return 0;
    if(strncmp(s, "/net/tls", 8) == 0)
        return 0;
    if(strncmp(s, "/srv/", 5) == 0 && ((mode&3) == OWRITE || (mode&3) == ORDWR))
        return 0;
    return 1;
}
```

⟨*function* update 86c⟩≡ (97c)

```
void
update(Rpc *rpc, vlong t)
{
    vlong t2;

    t2 = nsec();
    t = t2 - t;
    if(t < 0)
        t = 0;

    rpc->time += t;
    if(t < rpc->lo)
        rpc->lo = t;
    if(t > rpc->hi)
        rpc->hi = t;
}
```

⟨*function* Xversion 86d⟩≡ (97c)

```
void
Xversion(Fsrpc *r)
{
    Fcall thdr;
    vlong t;

    t = nsec();

    if(r->work.msize > IOHDRSZ+Maxfdata)
        thdr.msize = IOHDRSZ+Maxfdata;
    else
        thdr.msize = r->work.msize;
    myiounit = thdr.msize - IOHDRSZ;
    if(strncmp(r->work.version, "9P2000", 6) != 0){
        reply(&r->work, &thdr, Ebadver);
        r->busy = 0;
        return;
    }
    thdr.version = "9P2000";
```

```
    /* BUG: should clunk all fids */
    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tversion], t);
}
```

⟨*function* Xauth 87a⟩≡ (97c)

```
void
Xauth(Fsrpc *r)
{
    Fcall thdr;
    vlong t;

    t = nsec();

    reply(&r->work, &thdr, Enoauth);
    r->busy = 0;

    update(&stats->rpc[Tauth], t);
}
```

⟨*function* Xflush 87b⟩≡ (97c)

```
void
Xflush(Fsrpc *r)
{
    Fsrpc *t, *e;
    Fcall thdr;

    e = &Workq[Nr_workbufs];

    for(t = Workq; t < e; t++) {
        if(t->work.tag == r->work.oldtag) {
            DEBUG(2, "\tQ busy %d pid %p can %d\n", t->busy, t->pid, t->canint);
            if(t->busy && t->pid) {
                t->flushtag = r->work.tag;
                DEBUG(2, "\tset flushtag %d\n", r->work.tag);
                if(t->canint)
                    postnote(PNPROC, t->pid, "flush");
                r->busy = 0;
                return;
            }
        }
    }

    reply(&r->work, &thdr, 0);
    DEBUG(2, "\tflush reply\n");
    r->busy = 0;
}
```

⟨*function* Xattach 87c⟩≡ (97c)

```
void
Xattach(Fsrpc *r)
{
    Fcall thdr;
    Fid *f;
    vlong t;

    t = nsec();
```

```
    f = newfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }

    f->f = root;
    thdr.qid = f->f->qid;
    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tattach], t);
}
```

⟨*function* Xwalk 88⟩≡ (97c)

```
void
Xwalk(Fsrpc *r)
{
    char errbuf[ERRMAX], *err;
    Fcall thdr;
    Fid *f, *n;
    File *nf;
    vlong t;
    int i;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }
    n = nil;
    if(r->work.newfid != r->work.fid){
        n = newfid(r->work.newfid);
        if(n == 0) {
            reply(&r->work, &thdr, Edupfid);
            r->busy = 0;
            return;
        }
        n->f = f->f;
        f = n; /* walk new guy */
    }

    thdr.nwqid = 0;
    err = nil;
    for(i=0; i<r->work.nwname; i++){
        if(i >= MAXWELEM)
            break;
        if(strcmp(r->work.wname[i], "..") == 0) {
            if(f->f->parent == 0) {
                err = Exmnt;
                break;
            }
            f->f = f->f->parent;
            thdr.wqid[thdr.nwqid++] = f->f->qid;
            continue;
        }
```

```
        nf = file(f->f, r->work.wname[i]);
        if(nf == 0) {
            errstr(errbuf, sizeof errbuf);
            err = errbuf;
            break;
        }

        f->f = nf;
        thdr.wqid[thdr.nwqid++] = nf->qid;
        continue;
    }

    if(err == nil && thdr.nwqid == 0 && r->work.nwname > 0)
        err = "file does not exist";

    if(n != nil && (err != 0 || thdr.nwqid < r->work.nwname)){
        /* clunk the new fid, which is the one we walked */
        freefid(n->nr);
    }

    if(thdr.nwqid > 0)
        err = nil;
    reply(&r->work, &thdr, err);
    r->busy = 0;

    update(&stats->rpc[Twalk], t);
}
```

⟨*function* Xclunk 89⟩≡ (97c)

```
void
Xclunk(Fsrpc *r)
{
    Fcall thdr;
    Fid *f;
    vlong t;
    int fid;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }

    if(f->fid >= 0)
        close(f->fid);

    fid = r->work.fid;
    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tclunk], t);

    if(f->nread || f->nwrite)
        fidreport(f);

    freefid(fid);
```

```
}
```

⟨*function* Xstat 90a⟩≡ (97c)

```
void
Xstat(Fsrpc *r)
{
    char err[ERRMAX], path[128];
    uchar statbuf[STATMAX];
    Fcall thdr;
    Fid *f;
    int s;
    vlong t;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }
    makepath(path, f->f, "");
    if(!okfile(path, -1)){
        snprint(err, sizeof err, "iostats: can't simulate %s", path);
        reply(&r->work, &thdr, err);
        r->busy = 0;
        return;
    }

    if(f->fid >= 0)
        s = fstat(f->fid, statbuf, sizeof statbuf);
    else
        s = stat(path, statbuf, sizeof statbuf);

    if(s < 0) {
        errstr(err, sizeof err);
        reply(&r->work, &thdr, err);
        r->busy = 0;
        return;
    }
    thdr.stat = statbuf;
    thdr.nstat = s;
    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tstat], t);
}
```

⟨*function* Xcreate 90b⟩≡ (97c)

```
void
Xcreate(Fsrpc *r)
{
    char err[ERRMAX], path[128];
    Fcall thdr;
    Fid *f;
    File *nf;
    vlong t;

    t = nsec();
```

```
    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }


    makepath(path, f->f, r->work.name);
    f->fid = create(path, r->work.mode, r->work.perm);
    if(f->fid < 0) {
        errstr(err, sizeof err);
        reply(&r->work, &thdr, err);
        r->busy = 0;
        return;
    }

    nf = file(f->f, r->work.name);
    if(nf == 0) {
        errstr(err, sizeof err);
        reply(&r->work, &thdr, err);
        r->busy = 0;
        return;
    }

    f->mode = r->work.mode;
    f->f = nf;
    thdr.iounit = myiounit;
    thdr.qid = f->f->qid;
    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tcreate], t);
}
```

⟨*function* Xremove 91⟩≡ (97c)

```
void
Xremove(Fsrpc *r)
{
    char err[ERRMAX], path[128];
    Fcall thdr;
    Fid *f;
    vlong t;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }

    makepath(path, f->f, "");
    DEBUG(2, "\tremove: %s\n", path);
    if(remove(path) < 0) {
        errstr(err, sizeof err);
        reply(&r->work, &thdr, err);
        freefid(r->work.fid);
        r->busy = 0;
```

```
        return;
    }

    f->f->inval = 1;
    if(f->fid >= 0)
        close(f->fid);
    freefid(r->work.fid);

    reply(&r->work, &thdr, 0);
    r->busy = 0;

    update(&stats->rpc[Tremove], t);
}
```

⟨*function* Xwstat 92a⟩≡ (97c)

```
void
Xwstat(Fsrpc *r)
{
    char err[ERRMAX], path[128];
    Fcall thdr;
    Fid *f;
    int s;
    vlong t;

    t = nsec();

    f = getfid(r->work.fid);
    if(f == 0) {
        reply(&r->work, &thdr, Ebadfid);
        r->busy = 0;
        return;
    }
    if(f->fid >= 0)
        s = fwstat(f->fid, r->work.stat, r->work.nstat);
    else {
        makepath(path, f->f, "");
        s = wstat(path, r->work.stat, r->work.nstat);
    }
    if(s < 0) {
        errstr(err, sizeof err);
        reply(&r->work, &thdr, err);
    }
    else
        reply(&r->work, &thdr, 0);

    r->busy = 0;
    update(&stats->rpc[Twstat], t);
}
```

⟨*function* slave 92b⟩≡ (97c)

```
void
slave(Fsrpc *f)
{
    int r;
    Proc *p;
    uintptr pid;
    static int nproc;

    for(;;) {
        for(p = Proclist; p; p = p->next) {
```

```
            if(p->busy == 0) {
                f->pid = p->pid;
                p->busy = 1;
                pid = (uintptr)rendezvous((void*)p->pid, f);
                if(pid != p->pid)
                    fatal("rendezvous sync fail");
                return;
            }
        }

        if(++nproc > MAXPROC)
            fatal("too many procs");

        r = rfork(RFPROC|RFMEM);
        if(r < 0)
            fatal("rfork");

        if(r == 0)
            blockingslave();

        p = malloc(sizeof(Proc));
        if(p == 0)
            fatal("out of memory");

        p->busy = 0;
        p->pid = r;
        p->next = Proclist;
        Proclist = p;

        rendezvous((void*)p->pid, p);
    }
}
```

⟨*function* blockingslave 93⟩≡ (97c)

```
void
blockingslave(void)
{
    Proc *m;
    uintptr pid;
    Fsrpc *p;
    Fcall thdr;

    notify(flushaction);

    pid = getpid();

    m = rendezvous((void*)pid, 0);

    for(;;) {
        p = rendezvous((void*)pid, (void*)pid);
        if(p == (void*)~0)   /* Interrupted */
            continue;

        DEBUG(2, "\tslave: %p %F b %d p %p\n", pid, &p->work, p->busy, p->pid);
        if(p->flushtag != NOTAG)
            return;

        switch(p->work.type) {
        case Tread:
            slaveread(p);
```

```
            break;
        case Twrite:
            slavewrite(p);
            break;
        case Topen:
            slaveopen(p);
            break;
        default:
            reply(&p->work, &thdr, "exportfs: slave type error");
        }
        if(p->flushtag != NOTAG) {
            p->work.type = Tflush;
            p->work.tag = p->flushtag;
            reply(&p->work, &thdr, 0);
        }
        p->busy = 0;
        m->busy = 0;
    }
}
```

⟨*function* slaveopen 94⟩≡ (97c)

```
void
slaveopen(Fsrpc *p)
{
    char err[ERRMAX], path[128];
    Fcall *work, thdr;
    Fid *f;
    vlong t;

    work = &p->work;

    t = nsec();

    f = getfid(work->fid);
    if(f == 0) {
        reply(work, &thdr, Ebadfid);
        return;
    }
    if(f->fid >= 0) {
        close(f->fid);
        f->fid = -1;
    }

    makepath(path, f->f, "");
    DEBUG(2, "\topen: %s %d\n", path, work->mode);

    p->canint = 1;
    if(p->flushtag != NOTAG)
        return;

    if(!okfile(path, work->mode)){
        snprint(err, sizeof err, "iostats can't simulate %s", path);
        reply(work, &thdr, err);
        return;
    }

    /* There is a race here I ignore because there are no locks */
    f->fid = open(path, work->mode);
    p->canint = 0;
    if(f->fid < 0) {
```

```
        errstr(err, sizeof err);
        reply(work, &thdr, err);
        return;
    }

    DEBUG(2, "\topen: fd %d\n", f->fid);
    f->mode = work->mode;
    thdr.iounit = myiounit;
    thdr.qid = f->f->qid;
    reply(work, &thdr, 0);

    update(&stats->rpc[Topen], t);
}
```

⟨*function* slaveread 95⟩≡ (97c)

```
void
slaveread(Fsrpc *p)
{
    char data[Maxfdata], err[ERRMAX];
    Fcall *work, thdr;
    Fid *f;
    int n, r;
    vlong t;

    work = &p->work;

    t = nsec();

    f = getfid(work->fid);
    if(f == 0) {
        reply(work, &thdr, Ebadfid);
        return;
    }

    n = (work->count > Maxfdata) ? Maxfdata : work->count;
    p->canint = 1;
    if(p->flushtag != NOTAG)
        return;
    /* can't just call pread, since directories must update the offset */
    if(f->f->qid.type&QTDIR){
        if(work->offset != f->offset){
            if(work->offset != 0){
                snprint(err, sizeof err, "can't seek in directory from %lld to %lld", f->offset, work->offset);
                reply(work, &thdr, err);
                return;
            }
            if(seek(f->fid, 0, 0) != 0){
                errstr(err, sizeof err);
                reply(work, &thdr, err);
                return;
            }
            f->offset = 0;
        }
        r = read(f->fid, data, n);
        if(r > 0)
            f->offset += r;
    }else
        r = pread(f->fid, data, n, work->offset);
    p->canint = 0;
    if(r < 0) {
```

```
        errstr(err, sizeof err);
        reply(work, &thdr, err);
        return;
    }

    DEBUG(2, "\tread: fd=%d %d bytes\n", f->fid, r);

    thdr.data = data;
    thdr.count = r;
    stats->totread += r;
    f->nread++;
    f->bread += r;
    reply(work, &thdr, 0);

    update(&stats->rpc[Tread], t);
}
```

⟨*function* slavewrite 96⟩≡ (97c)

```
void
slavewrite(Fsrpc *p)
{
    char err[ERRMAX];
    Fcall *work, thdr;
    Fid *f;
    int n;
    vlong t;

    work = &p->work;

    t = nsec();

    f = getfid(work->fid);
    if(f == 0) {
        reply(work, &thdr, Ebadfid);
        return;
    }

    n = (work->count > Maxfdata) ? Maxfdata : work->count;
    p->canint = 1;
    if(p->flushtag != NOTAG)
        return;
    n = pwrite(f->fid, work->data, n, work->offset);
    p->canint = 0;
    if(n < 0) {
        errstr(err, sizeof err);
        reply(work, &thdr, err);
        return;
    }

    DEBUG(2, "\twrite: %d bytes fd=%d\n", n, f->fid);

    thdr.count = n;
    f->nwrite++;
    f->bwrite += n;
    stats->totwrite += n;
    reply(work, &thdr, 0);

    update(&stats->rpc[Twrite], t);
}
```

⟨*function* reopen 97a⟩≡ (97c)

```
void
reopen(Fid *f)
{
    USED(f);
    fatal("reopen");
}
```

⟨*function* flushaction 97b⟩≡ (97c)

```
void
flushaction(void *a, char *cause)
{
    USED(a);
    if(strncmp(cause, "kill", 4) == 0)
        noted(NDFLT);

    noted(NCONT);
}
```

⟨iostats/statsrv.c 97c⟩≡

```
#include <u.h>
#include <libc.h>
#include <auth.h>
#include <fcall.h>

#include "statfs.h"

⟨global Ebadfid 85a⟩
⟨global Enotdir 85b⟩
⟨global Edupfid 85c⟩
⟨global Eopen 85d⟩
⟨global Exmnt 85e⟩
⟨global Enoauth 85f⟩
⟨global Ebadver 86a⟩

⟨function okfile 86b⟩

⟨function update 86c⟩

⟨function Xversion 86d⟩

⟨function Xauth 87a⟩

⟨function Xflush 87b⟩

⟨function Xattach 87c⟩

⟨function Xwalk 88⟩

⟨function Xclunk 89⟩

⟨function Xstat 90a⟩

⟨function Xcreate 90b⟩


⟨function Xremove 91⟩

⟨function Xwstat 92a⟩
```

⟨*function* `slave` 92b⟩

⟨*function* `blockingslave` 93⟩

⟨*function* `slaveopen` 94⟩

⟨*function* `slaveread` 95⟩

⟨*function* `slavewrite` 96⟩

⟨*function* `reopen` 97a⟩

⟨*function* `flushaction` 97b⟩

Glossary

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. For an introduction see http://en.wikipedia.org/wiki/Literate_Program. cited page(s) 4

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 4